

Full-stack engineering through projects

54 complete project briefs

JavaScript -> TypeScript | From first program to independent delivery

10-15 hours each week

120 planned study weeks / 1,200-1,800 hours

First portfolio review: Project 25 / week 59

Build one product in increasingly demanding slices. Reuse earlier work. Induce failures, diagnose them, and show evidence that the repair holds.

Every brief includes the purpose, new and reinforced topics, requirements, constraints, failure drills, completion checks, optional stretch work, search terms, and report-back evidence.

This edition contains the entire roadmap now. The next assignment can still change when your results reveal a gap.

Prepared 15 September 2026

No topic is marked learned merely because it appears in this plan.

How to use this curriculum

Your starting point

Zero coding experience; JavaScript then TypeScript; 10-15 hours per week; no existing proof of work. The goal is evidence of full-stack delivery, debugging, and explanation. macOS is the starting assumption. Project 1 can use the already available Node.js 24.14.0 and Git; later projects must record and pin the supported versions selected when they begin.

The product path

Projects 1-52 build StudyTrack, from a calculator to a shared study platform. Project 3 starts a browser surface in the same repository and later absorbs the earlier logic. Project 53 starts a separate helpdesk repository to test transfer; Project 54 ships it. All other projects extend earlier work. Keep prior useful behavior and regression checks unless the brief explicitly replaces a surface.

Time and milestones

The base plan is 120 study weeks, or 1,200-1,800 hours. Projects 1-25 total 59 weeks, or 590-885 hours, and end with the first portfolio/application-readiness review. The remaining projects total 61 weeks. These are planning estimates, not deadlines or a hiring guarantee; review and reteaching can extend the calendar.

How to use this complete edition

All 54 briefs are assigned here in full. Work through them sequentially and use each report-back section as the handoff for review. For a typical week, allow 2-3 hours for targeted reading, 6-9 for building, and 2-3 for verification and explanation; adjust within your 10-15-hour total. Read only the concepts needed for the current requirement.

What counts as a topic

Each New topics bullet is one bounded learning focus mapped to a syllabus item. Its subterms belong to the same small exercise. Projects list at most three new focuses and two to four earlier areas to stress harder. Project 1 is the necessary exception: there is no prior work. Zero-new-topic projects are deliberate integration or transfer assessments.

How to use this curriculum

How the spiral is assessed

The coverage map identifies two to four assessment touchpoints for each listed area; routine use can continue beyond them. When a row groups techniques, their first relevant introduction may occur at different projects in the sequence. Read the individual brief for the exact focus. A package installation or tutorial completion does not demonstrate a concept.

If a concept does not land

Report the failing case, your explanation, and the help used. The next project replaces one or more new-topic slots with a different-angle exercise on that gap and reuses it in the product; displaced material moves later. The project remains within its weekly size. Repeat the acceptance test and an unseen variant before advancing the topic status. An unresolved prerequisite pauses dependent new work, not all useful practice.

Scope, stack, and budget

PostgreSQL, React/Next.js, MongoDB, Redis, and the named infrastructure tools are sensible defaults for these briefs. Library/provider decisions can change when you reach them. External accounts and spending are discussed at that point; this document authorizes no purchases. Use local labs for infrastructure by default and run only the services needed for the active slice. A local deployment plus reproducible demo remains an option where public hosting is unavailable.

The working agreement

Build your own application code. Ask for requirements clarification, hints, and review; request short illustrative code only when stuck. Keep experiments and faults in owned, disposable environments. Optional stretch goals do not block completion or count as required topic coverage. Document fault, symptom, evidence, repair, and verification for every deliberate break.

Review and completion

Every requirement and Definition of done item is mandatory within the stated scope. Report evidence, not just a screenshot of the homepage. The mentor checks correctness, fragile assumptions, independence, and integration with earlier work. Core readiness is assessed at Project 25; the advanced platform syllabus can continue alongside applications and interview preparation.

Full roadmap

E = extends project. G = greenfield. Page numbers link to the full brief.

#	Project	Weeks / mode	Page
1	Study-time calculator	3 / G	7
2	Study log that survives restarts	3 / E1	9
3	Public study profile	2 / G	11
4	First website on the internet	2 / E3	13
5	Interactive weekly planner	2 / E4 + P1	15
6	Shared study log over HTTP	3 / E5 + P2	17
7	API another developer can use	2 / E6	19
8	Reliable records in PostgreSQL	3 / E7	21
9	Fast, correct progress reports	2 / E8	23
10	Convert safely to TypeScript	2 / E9	25
11	React study dashboard	3 / E10	27
12	Multi-screen planner with reliable state	2 / E11	29
13	Defend public inputs	2 / E12	31
14	Private accounts and permissions	3 / E13	33
15	Sign in through an identity provider	2 / E14	35
16	Smooth, accessible dashboard	2 / E15	37
17	Shareable server-rendered pages	3 / E16	39
18	Fast public course catalogue	2 / E17	41

Full roadmap

E = extends project. G = greenfield. Page numbers link to the full brief.

#	Project	Weeks / mode	Page
19	Fresh pages without repeated work	2 / E18	43
20	Releases that check themselves	2 / E19	45
21	Reproducible container release	2 / E20	47
22	Deploy to a Linux server	3 / E21	49
23	Diagnose an unreachable or overloaded site	2 / E22	51
24	Explain production failures	2 / E23	53
25	First portfolio release and review	3 / E24	55
26	Shared sessions and fair API limits	2 / E25	57
27	Survive a stale or unavailable cache	2 / E26	59
28	Generate reports in the background	2 / E27	61
29	Reliable webhook delivery	2 / E28	63
30	Live study-room activity	2 / E29	65
31	Flexible dashboard data and access	2 / E30	67
32	Flexible learning-resource library	2 / E31	69
33	Search resources by intent	2 / E32	71
34	Repair stale search and build usage reports	2 / E33	73
35	Follow a request across processes	2 / E34	75
36	Alerts that identify actionable failures	2 / E35	77

Full roadmap

E = extends project. G = greenfield. Page numbers link to the full brief.

#	Project	Weeks / mode	Page
37	Activity events across two services	3 / E36	79
38	Recover missing and duplicate events	2 / E37	81
39	One entrance for multiple services	2 / E38	83
40	Shared packages with clear contracts	2 / E39	85
41	Find a blocked event loop and memory leak	2 / E40	87
42	Keep live rooms responsive under pressure	2 / E41	89
43	Reusable dashboard styling	2 / E42	91
44	Two alternative component styling approaches	2 / E43	93
45	Choose and consolidate the styling system	1 / E44	95
46	Run the platform on local Kubernetes	3 / E45	97
47	Recover from bad rollouts and dead pods	2 / E46	99
48	Make the repository maintainable by someone else	2 / E47	101
49	Explain and reduce page-loading cost	2 / E48	103
50	Tenant isolation and hostile authorization tests	2 / E49	105
51	Upgrade APIs without breaking old clients	2 / E50	107
52	Full platform failure-and-recovery exercise	2 / E51	109
53	Independent helpdesk product	3 / G	111
54	Ship and defend the helpdesk release	3 / E53	113

PROJECT 1 - Study-time calculator

Size: 3 week(s), 30-45 hours total | Mode: Greenfield - your first program.

Why now

You need to turn a small requirement into a program, run it yourself, and preserve changes you can explain. This connects those activities before adding a browser or database.

New topics

- Common > Fundamentals > OS and CLI: directories, paths, files, and running commands.
- Frontend > JS / TS / Node: values, conditions, functions, arguments, and running JavaScript in Node.
- Common > VCS and hosting > Git: working directory, staging, commits, diffs, and history.

Reinforced topics

- None yet. Project 1 is the necessary exception to the earlier-project requirement; repeat change, run, inspect, and commit within this assignment.

The build

A terminal calculator that tells a learner how many weeks a study goal will take, including the hours needed in the final week.

Requirements

1. Create studytrack with planner.js, README.md, and a local Git repository. The command format is: `node planner.js <total-hours> <weekly-hours>`; for example, `node planner.js 30 12`.
2. Accept exactly two arguments: total required hours, then available hours per week. Both must be positive whole numbers; reject missing/extra arguments, zero, negatives, fractions, and nonnumeric text.
3. Print total hours, weekly hours, required whole weeks, and final-week hours. Partial weeks count as whole required weeks; exact multiples must not show zero final-week hours.
4. Keep calculation in a named function, separate from printing. Invalid input produces a clear error and usage example, without a plan.
5. Pass these valid cases: 30/12 -> 3 weeks, final 6 hours; 24/12 -> 2, final 12; 5/12 -> 1, final 5; 1/1 -> 1, final 1.
6. Reject these argument sets: 0 12; 30 0; -5 12; 30 2.5; 30 twelve; 30 12hours; 30; 30 12 4.
7. Make at least four meaningful commits. Document prerequisites, command, input rules, and a valid example; verify them from a fresh local clone.

PROJECT 1 - Study-time calculator

Constraints, verification and report

Constraints

- No npm packages or frameworks: expose what JavaScript and Node provide.
- No UI, persistence, or accounts: complete one input-to-answer path.
- Use terminal commands for running and basic Git work; do not hide paths or repository state.
- No copied/generated complete solution. Use documentation and hints, and explain every submitted line.

Break it on purpose

- Temporarily round weeks down. Observe the wrong answer for 30 12, explain why it fails, repair it, and rerun partial and exact-week cases.
- Run the usual command from the parent folder. Explain where Node looked after the missing-file error; repair the path or current directory.

Definition of done

- [] All four valid and eight invalid cases pass.
- [] Both failures have observation, explanation, repair, and verification notes.
- [] Calculation and printing are separate; every line can be explained.
- [] Four meaningful commits exist, Git status is clean, and a fresh local clone runs from the README.

Stretch goals

- Optional: add --help.
- Optional: use singular 'week' and 'hour' correctly.

Concepts to read up on

- current working directory; relative and absolute paths
- Node process.argv; strings versus numbers; Number.isInteger
- functions; conditions; rounding up; remainder
- Git working tree, staging, commit, diff, log, clone

Report back with

- Repository path or link; planner.js and README.md.
- Output for all twelve acceptance examples; git log --oneline and git status.
- Both failure reports; identify any help you used.
- Explain why 30/12 needs three weeks, how '12' differs from 12, and what staging changes versus committing.

Suggested pacing

Week 1: CLI, tiny JavaScript experiments, and commits. Week 2: calculations and validation. Week 3: failure drills, acceptance cases, and fresh-clone verification.

PROJECT 2 - Study log that survives restarts

Size: 3 week(s), 30-45 hours total | Mode: Extends Project 1.

Why now

A calculator forgets everything when it exits. Saving a small history makes data modelling and testable functions useful immediately.

New topics

- Frontend > JS / TS / Node: arrays/objects, modules, JSON, and one-process file persistence.
- Common > Testing and performance: unit tests for pure calculation and validation functions.
- Common > Documentation: a README another person can follow.

Reinforced topics

- P1 functions: reuse the calculator without copying its logic.
- P1 CLI: support multiple commands and a data-file path.
- P1 Git: separate extraction, storage, and testing into reviewable commits.

The build

A learner adds study sessions and sees completed hours and an updated estimate of the weeks remaining.

Requirements

1. Keep the P1 calculator command working. Add add <minutes> <note> and summary <goal-hours> <weekly-hours>; use positive integer minutes and a nonempty note.
2. Save sessions as JSON with an ID, minutes, note, and timestamp; show accumulated minutes accurately across three separate process runs.
3. Store calculation, storage, and command handling in separate modules. Convert minutes to hours deliberately and state the display rounding rule.
4. Handle an absent data file as an empty log; handle invalid JSON as a visible error without overwriting it.
5. Add unit tests for the P1 boundary cases, minute totals, empty history, and a completed goal.
6. Document setup, commands, data location, and how to run tests; verify from a fresh clone.

PROJECT 2 - Study log that survives restarts

Constraints, verification and report

Constraints

- No database or third-party packages: see the file boundary first.
- One running writer only: concurrency is explicitly out of scope until P6/P8.
- No happy-path-only tests: boundary behavior is part of the contract.

Break it on purpose

- Corrupt a disposable copy of the JSON file. Observe the parse failure, preserve the bad file, restore a known-good copy, and verify the original session count.

Definition of done

- [] Data survives restarts and existing calculator cases still pass.
- [] Tests cover empty, partial, and completed goals.
- [] Corrupt data is not silently replaced.
- [] A fresh clone works using only the README.

Stretch goals

- Optional: list the latest five sessions.

Concepts to read up on

- JavaScript arrays objects modules
- JSON parse stringify; Node file system
- pure functions; Node test runner; assertions
- README prerequisites and reproducible examples

Report back with

- Repository; calculation, storage, CLI, and test files.
- Three-run transcript and test output.
- Corrupt-file failure report and fresh-clone notes.
- Explain why a unit test should not depend on your personal data file.

PROJECT 3 - Public study profile

Size: 2 week(s), 20-30 hours total | Mode: Greenfield - a small website teaches the browser document model without carrying terminal-specific structure. Keep it in the same repository under web/.

Why now

You can now build a small program, but browser interfaces have their own structure. A single public page introduces that structure before browser scripting.

New topics

- Frontend > CSS: semantic HTML for headings, navigation, forms, and lists.
- Frontend > CSS: cascade, box model, responsive layout with one layout technique.
- Common > Testing and performance: keyboard and basic accessibility checks.

Reinforced topics

- P1 CLI: create and navigate a second runnable artifact.
- P1-P2 Git: review structural and visual changes separately.

The build

A public study profile showing a learner's goal, weekly availability, three resources, and a sample weekly plan.

Requirements

1. Use header, navigation, main content, and footer landmarks with one descriptive page title and a logical heading order.
2. Show the goal, availability, resources, and sample plan using appropriate HTML; links must have meaningful text.
3. Create a responsive layout that works at 360px and 1280px without horizontal scrolling.
4. Include a labelled, non-submitting sample goal form; clearly mark it as a preview until P5.
5. Ensure all links and form controls are keyboard reachable with visible focus; verify text contrast with a browser tool.
6. Document the page-opening instructions and keyboard check results.

PROJECT 3 - Public study profile

Constraints, verification and report

Constraints

- No JavaScript, CSS framework, or copied template: learn structure and cascade.
- No layout tables or absolute positioning for the overall page: practice normal flow and layout.
- No placeholder-only labels: accessible names must remain visible.

Break it on purpose

- Remove a form label and a focus style on a temporary branch. Observe the ambiguity during keyboard navigation, restore both, and repeat the same route through the page.

Definition of done

- [] Required content and semantic structure are present.
- [] Both viewport checks pass.
- [] Keyboard operation and visible labels pass.
- [] The README and before/after screenshots explain the accessibility repair.

Stretch goals

- Optional: support a reduced-motion preference if you add animation.

Concepts to read up on

- semantic HTML landmarks headings labels
- CSS cascade specificity box model flexbox
- responsive design; keyboard focus; contrast

Report back with

- Repository; web HTML and CSS files.
- 360px and 1280px screenshots; keyboard-check notes.
- Failure report.
- Explain why a visual heading is not necessarily an HTML heading.

PROJECT 4 - First website on the internet

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 3.

Why now

Opening a local file hides the request and deployment path. Publishing the same small page lets you trace a real browser request without backend complexity.

New topics

- Common > Fundamentals > Networking: HTTP/HTTPS, URLs, DNS, requests, responses, and status codes.
- Common > Dev environment and deployment: publishing a static artifact.
- Common > VCS and hosting > GitHub: remote branches, a pull request, and merge workflow.

Reinforced topics

- P3 HTML/CSS: correct relative resource URLs after publication.
- P1 Git: compare local commits with the remote branch.
- P2 README: provide a reproducible publishing and rollback procedure.

The build

A publicly reachable study profile with a working browser-to-host request path.

Requirements

1. Publish the P3 static page using a static host; GitHub Pages is the default. A provider subdomain is sufficient.
2. Make the publication change on a branch, open a PR, inspect its diff, then merge it.
3. Record the hostname's DNS answer, the document's HTTP status, and the content type of one stylesheet using browser tools or a command-line client.
4. Verify HTTPS, all resource links, and both P3 viewport sizes at the public URL.
5. Record the deployed commit and prove you can restore the previous known-good page.

PROJECT 4 - First website on the internet

Constraints, verification and report

Constraints

- No custom domain purchase or backend: publication must stay small.
- No treating a successful upload as verification: inspect the public page.
- No claiming DNS returns a URL: distinguish names, addresses, and HTTP paths.

Break it on purpose

- Publish a deliberately wrong stylesheet path. Observe its 404 in the network panel and the unstyled page; repair the path, redeploy, and verify a 200 response.

Definition of done

- [] Public HTTPS page works with no required-resource failures.
- [] PR and deployed commit are identifiable.
- [] DNS and HTTP evidence is recorded.
- [] Broken-asset diagnosis and rollback are demonstrated.

Stretch goals

- Optional: add a useful static 404 page.

Concepts to read up on

- DNS resolution; URL hostname path
- HTTP request response status content-type
- HTTPS certificate overview
- GitHub branches pull requests; static hosting

Report back with

- Public URL, repository, and PR.
- Network-panel evidence for document and stylesheet; DNS output.
- Deployment/rollback notes and failure report.
- Explain which part DNS resolves and which part the web server handles.

PROJECT 5 - Interactive weekly planner

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 4 and absorbs Project 1's calculation module.

Why now

Your page is visible but cannot respond to the learner. Connecting a form to existing logic exposes browser events and rendering without adding a framework.

New topics

- Frontend > JS / TS / Node: DOM events and updates.
- Frontend > JS / TS / Node: closures and event-handler state.
- Frontend > How browsers work: rendering path and reflow versus repaint.

Reinforced topics

- P1 functions: reuse calculations in a different runtime.
- P3 semantic HTML: turn the preview into a working labelled form.
- P3 CSS: support result and error states without layout breakage.
- P4 deployment: ship browser modules with correct paths.

The build

Visitors enter a study goal and weekly hours and immediately receive an accessible plan.

Requirements

1. Reuse the pure calculation module in the browser; do not import Node-only modules into it.
2. Handle form submission without a page reload and apply P1's input rules.
3. Show an inline error associated with the relevant field; announce an updated result accessibly.
4. Add a reset button that clears values, errors, and results; verify five repeated submit/reset cycles.
5. Keep one event handler per action; explain a closure used by a handler.
6. Capture one browser performance recording and identify a DOM change that triggers layout and a visual change that only needs painting.

PROJECT 5 - Interactive weekly planner

Constraints, verification and report

Constraints

- No React, form library, or browser storage: expose browser primitives.
- No inline event attributes or duplicate listener registration: make lifecycle visible.
- No inserting user input as raw HTML: display it as text.

Break it on purpose

- Register the submit handler twice on a temporary branch. Observe duplicate work with a counter or log, locate the duplicate registration, and verify one execution per submit after repair.

Definition of done

- [] All P1 input cases work in the browser.
- [] Repeated submit/reset cycles remain correct.
- [] Keyboard and error announcements are checked.
- [] Public deployment and rendering evidence are current.

Stretch goals

- Optional: add a preset-hours button without duplicating calculation logic.

Concepts to read up on

- DOM addEventListener preventDefault
- closures lexical scope
- textContent versus HTML insertion
- critical rendering path; reflow repaint

Report back with

- Live URL; shared calculation module and browser script.
- Short recording of valid, invalid, and reset behavior.
- Performance capture and duplicate-handler report.
- Explain why Node's process.argv cannot be used directly in this browser module.

PROJECT 6 - Shared study log over HTTP

Size: 3 week(s), 30-45 hours total | Mode: Extends Project 5 and absorbs Project 2's storage module.

Why now

A browser-only planner cannot share persistent state with another browser. One small server connects the interface to storage and makes asynchronous failures visible.

New topics

- Frontend > JS / TS / Node: the native Node HTTP server runtime.
- Backend > APIs: REST resources and request/response contracts.
- Frontend > JS / TS / Node: promises, async/await, concurrency, and an initial event-loop model.

Reinforced topics

- P2 modules: keep storage separate from HTTP handling.
- P2 persistence: detect overlapping writes instead of losing records.
- P4 HTTP: use meaningful methods, statuses, and media types.
- P5 UI states: show loading, empty, success, and failure states.

The build

Two browser tabs add study sessions to one shared server-side log.

Requirements

1. Serve the UI and API from one origin. Implement GET /api/sessions, POST /api/sessions, and GET /health.
2. Accept only the documented session fields, reject malformed JSON and oversized bodies, and return consistent JSON errors.
3. Make the browser list and add records through fetch; prevent accidental double submission while a request is pending.
4. Persist successful writes before acknowledging success. For this single-process file store, reject overlapping mutations with a documented retryable response rather than silently losing data.
5. Verify two tabs see the same records after refresh and records survive a server restart.
6. Keep the working local version reproducible. Remote deployment of the Node service is deferred to P22.

PROJECT 6 - Shared study log over HTTP

Constraints, verification and report

Constraints

- No Express or framework router: inspect method, path, headers, and body yourself.
- One server process and one writer: do not pretend the file store supports distributed concurrency.
- No blocking file operations inside request handlers: practice promise-based I/O.

Break it on purpose

- Stop the server while the page is open. Observe fetch failure, display a recoverable message instead of an endless spinner, restart the server, and retry without adding a duplicate record.

Definition of done

- [] All three endpoints follow their documented contracts.
- [] Two-tab and restart checks pass.
- [] Invalid JSON and overlapping writes have defined outcomes.
- [] Server-down recovery works and prior calculation tests remain green.

Stretch goals

- Optional: add deletion of a session with a deliberate confirmation in the UI.

Concepts to read up on

- Node http request response
- REST resource method status; JSON content type
- fetch async await rejection
- concurrent requests versus sequential operations

Report back with

- Repository; server, storage, and browser API files.
- HTTP examples for success, malformed input, and overlap.
- Two-tab/restart evidence and outage report.
- Explain the difference between a rejected promise and an HTTP error response.

PROJECT 7 - API another developer can use

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 6.

Why now

Manual checks do not protect an API as it changes. A documented contract and repeatable checks let another developer use it without reading every handler.

New topics

- Common > Testing and performance: HTTP integration tests.
- Frontend > Tooling / Common > Dev environment: npm scripts, dependency lockfiles, linting, and formatting as one toolchain setup.
- Common > Documentation: API documentation with executable request examples.

Reinforced topics

- P6 REST: keep response shape and status behavior consistent.
- P2 unit tests: retain fast calculation checks alongside integration checks.
- P1 validation: cover malformed and missing fields at the network boundary.
- P4 GitHub workflow: review a PR with evidence.

The build

The existing study API becomes a small service with a repeatable verification command and a clear consumer guide.

Requirements

1. Add scripts for starting, unit tests, integration tests, linting, and formatting checks; commit the lockfile.
2. Run integration tests against an isolated temporary data file and an ephemeral test server; never modify personal records.
3. Test create/list persistence, invalid JSON, invalid fields, unsupported method, unknown route, and content type.
4. Document each endpoint's request, success/error examples, status codes, and current single-process limitations.
5. Verify installation and all checks from a fresh clone using the locked dependency install.
6. Open a PR whose description states behavior changed and evidence collected.

PROJECT 7 - API another developer can use

Constraints, verification and report

Constraints

- No mocking the HTTP server or file boundary in integration tests: exercise actual integration.
- No adding endpoint features: this assignment improves confidence and usability.
- No formatter-wide changes mixed with behavior fixes: keep review understandable.

Break it on purpose

- Temporarily make an invalid request return a success status. Confirm an integration test fails for the contract violation, restore the correct status, and rerun the suite.

Definition of done

- [] Fresh-clone commands pass.
- [] Integration data is isolated and cleaned up.
- [] Every documented endpoint has success and failure evidence.
- [] The deliberately broken contract is caught automatically.

Stretch goals

- Optional: export a small machine-readable API specification.

Concepts to read up on

- integration test isolation; temporary directories
- npm scripts lockfile npm ci
- ESLint formatter
- API documentation request response examples

Report back with

- Repository and PR; package manifest, integration tests, and API guide.
- Fresh-clone check output and failing-then-passing test output.
- Explain what this integration test can catch that the calculator unit tests cannot.

PROJECT 8 - Reliable records in PostgreSQL

Size: 3 week(s), 30-45 hours total | Mode: Extends Project 7.

Why now

The file store cannot safely handle realistic concurrent mutations. A small relational schema introduces stronger guarantees while preserving the existing UI and API.

New topics

- Backend > Databases > SQL: schema design, constraints, and relationships.
- Backend > Databases > SQL: queries and joins.
- Backend > Databases > SQL: transactions and concurrent writes.

Reinforced topics

- P2 persistence: migrate existing sessions without changing their meaning.
- P6 REST: preserve consumer-visible behavior when storage changes.
- P7 integration tests: exercise a disposable real database.
- P1 validation: distinguish application errors from database constraints.

The build

Learners group study sessions into goals, with records stored reliably in PostgreSQL.

Requirements

1. Create goals and sessions tables with primary keys, a foreign key, positive-duration constraints, and timestamps; author versioned SQL migration files.
2. Implement raw parameterized SQL for create/list sessions and one goal-with-total query.
3. Provide a one-time import for P2 JSON and compare source and destination counts and total minutes.
4. Add one operation that creates a goal and its first session atomically; invalid session data must leave neither row behind.
5. Run ten overlapping valid session writes and verify all ten records persist.
6. Make database integration tests create clean test state and leave development records untouched.

PROJECT 8 - Reliable records in PostgreSQL

Constraints, verification and report

Constraints

- No ORM or query builder: write SQL and read the returned rows.
- No manual-only schema edits: another checkout must reproduce the schema.
- No cached total column yet: compute totals from authoritative session rows.

Break it on purpose

- Induce an error in the second step of goal-plus-session creation. Observe the attempted partial update, ensure the transaction rolls back, and prove neither row remains.

Definition of done

- [] Migrations rebuild a clean database.
- [] Import totals match and existing API tests pass.
- [] Concurrent writes persist without loss.
- [] Rollback has direct SQL evidence.

Stretch goals

- Optional: add a report joining goals and sessions, including goals with zero sessions.

Concepts to read up on

- PostgreSQL primary foreign check constraints
- parameterized SQL joins aggregation
- transactions commit rollback
- database migration and test database

Report back with

- Schema/migrations, SQL access module, import script, and tests.
- Import reconciliation and concurrent-write output.
- Transaction failure report.
- Explain why a foreign key and an application validation check solve different problems.

PROJECT 9 - Fast, correct progress reports

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 8.

Why now

A correct query can become painfully slow as data grows. You need to measure a concrete report and understand why an index helps or does not help.

New topics

- Backend > Databases > SQL: indexes.
- Backend > Databases > SQL: query plans with EXPLAIN ANALYZE.
- Common > Testing and performance: a bounded load test and profiling baseline.

Reinforced topics

- P8 SQL: aggregate sessions by goal and date correctly.
- P8 transactions: preserve correctness during simultaneous recording.
- P7 API contracts: keep the same result while improving execution.

The build

A learner opens a date-filtered progress report over a large synthetic study history.

Requirements

1. Add one report endpoint and one UI view showing totals for a selected goal and date range; verify empty and boundary-date cases.
2. Generate a deterministic synthetic dataset of at least 50,000 sessions, with enough distribution to make filtering meaningful.
3. Record an unindexed plan and runtime, add one justified index, and compare the plan and results.
4. Run a fixed five-client, 60-second read workload before and after; record machine, dataset, throughput, errors, and p50/p95 latency.
5. Explain index write/storage cost and why the chosen ordering matches the query.
6. Keep correctness assertions separate from timing thresholds; record measurements rather than inventing a universal latency guarantee.

PROJECT 9 - Fast, correct progress reports

Constraints, verification and report

Constraints

- No Redis or extra hardware: improve the query before introducing another system.
- No claiming an index works only because it exists: inspect the plan.
- No production data or unbounded traffic: use a local synthetic workload.

Break it on purpose

- Remove the chosen index on the test database. Observe the changed plan and workload result, restore it through a migration, and verify unchanged report totals.

Definition of done

- [] Date boundaries and empty reports are correct.
- [] Before/after plans and reproducible workload results exist.
- [] Index removal and restoration explain the performance difference.
- [] Existing integration checks still pass.

Stretch goals

- Optional: compare a second plausible index and reject it with evidence.

Concepts to read up on

- EXPLAIN ANALYZE sequential index scan
- composite index selectivity
- p50 p95 latency throughput
- load test warm-up and reproducibility

Report back with

- Report query, index migration, seed generator, and workload configuration.
- Plans and before/after measurements.
- Failure report.
- Explain one situation where the planner should prefer a sequential scan.

PROJECT 10 - Convert safely to TypeScript

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 9.

Why now

As data crosses more boundaries, accidental shape changes become harder to track. TypeScript can catch a useful class of mistakes while runtime validation still protects external input.

New topics

- Frontend > JS / TS / Node: TypeScript annotations, unions, and narrowing.
- Frontend > Tooling: transpilation and a browser bundle with a recorded build configuration.

Reinforced topics

- P2 modules: publish clear inputs and outputs between files.
- P7 unit/integration tests: protect behavior during conversion.
- P6 API contracts: distinguish parsed input from trusted application values.

The build

The same study tracker gains explicit data contracts and a reproducible browser build without changing its user-facing features.

Requirements

1. Convert the shared calculator and session model first, then the small API and browser client; keep behavior unchanged.
2. Enable strict checking and make type checking a separate command from bundling.
3. Represent loading/success/error states as a discriminated union and demonstrate exhaustive handling.
4. Treat incoming JSON as unknown until validated; add one runtime invalid-payload test that TypeScript alone cannot prevent.
5. Create development and production build commands with one bundler; record source, compiled output, and source-map locations.
6. Pass all existing checks from a locked fresh install.

PROJECT 10 - Convert safely to TypeScript

Constraints, verification and report

Constraints

- No broad any, ignore directives, or unsafe casts to silence errors: model the boundary.
- No simultaneous framework migration: isolate language/toolchain effects.
- No committing generated dependency directories: reproduce them from the lockfile.

Break it on purpose

- Change a shared field's type and observe a compiler failure in its consumer; restore it. Separately send a wrong-shaped JSON request and show runtime validation still rejects it.

Definition of done

- [] Strict type checking and production build pass.
- [] Existing behavior tests pass.
- [] Compile-time and runtime failures are demonstrated separately.
- [] The README documents the build and execution paths.

Stretch goals

- Optional: add a small generic result type used by two functions.

Concepts to read up on

- TypeScript strict unknown narrowing discriminated union
- type erasure runtime validation
- transpiler bundler source map

Report back with

- Type configuration, shared types, validation module, and build configuration.
- Type-error example, invalid-request result, and full check output.
- Explain why a successful type check does not prove a network payload is valid.

PROJECT 11 - React study dashboard

Size: 3 week(s), 30-45 hours total | Mode: Extends Project 10; replace the browser view while retaining the API and shared calculations.

Why now

Manual DOM updates get difficult as several controls affect the same screen. React introduces a rendering model that can express the dashboard from data and state.

New topics

- Frontend > React: components, rendering, state hooks, and controlled inputs.
- Frontend > Rendering strategies: client-side rendering.

Reinforced topics

- P1/P10 functions and types: keep calculations outside visual components.
- P3 CSS: preserve responsive layout and visible focus.
- P6 REST: consume the existing API contract.
- P5 UI states: express empty, loading, success, and failure declaratively.

The build

A React dashboard lets learners record a session, see history, and estimate remaining study time.

Requirements

1. Implement the existing form, session list, and total summary as a small component tree.
2. Use stable record IDs as list keys and controlled form fields with existing validation.
3. Reuse shared calculations and the API; do not duplicate totals in conflicting state variables.
4. Show all four API states and prevent duplicate submission while a save is pending.
5. Verify keyboard behavior and mobile layout match or improve on P5.
6. Explain one state update from event to next render; preserve previous unit and API checks.

PROJECT 11 - React study dashboard

Constraints, verification and report

Constraints

- No global state library, component kit, or generated dashboard: learn React's model.
- No manual DOM mutation for normal rendering.
- No new backend features: focus on the rendering transition.

Break it on purpose

- Use array positions as keys, then reorder records while a row has local UI state. Observe state following the wrong row, restore stable IDs, and verify correct behavior.

Definition of done

- [] All three dashboard features work against the existing API.
- [] Correctness survives reordering and failed saves.
- [] Responsive and keyboard checks pass.
- [] The component/state explanation matches the implementation.

Stretch goals

- Optional: add a presentational empty-state component reused twice.

Concepts to read up on

- React render state props controlled input
- list keys identity
- derived state; client-side rendering

Report back with

- Live or locally reproducible demo; component tree and shared logic.
- Before/after key-defect evidence and check output.
- Explain which values are source state and which are derived.

PROJECT 12 - Multi-screen planner with reliable state

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 11.

Why now

More than one view introduces state ownership and lifecycle problems. A small navigation flow makes state requests and lost edits observable.

New topics

- Frontend > React: effects, cleanup, and lifecycle.
- Frontend > React: context and reducer-based shared state.
- Common > Testing and performance: end-to-end browser tests.

Reinforced topics

- P11 rendering: choose component boundaries by state ownership.
- P10 TypeScript: model state transitions explicitly.
- P2 unit testing: test reducer transitions as pure behavior.
- P3 accessibility: preserve focus and keyboard navigation.

The build

Learners move between goal overview and goal detail while filters and save status behave predictably.

Requirements

1. Add overview and detail views using simple in-app navigation; URL routing is not required yet.
2. Place only genuinely shared state in a context/reducer; keep temporary form values local.
3. Cancel or ignore obsolete requests when the selected goal changes.
4. Clean up effects on unmount and demonstrate no duplicate subscriptions during development checks.
5. Add E2E tests for creating a session, switching goals, and recovering from a failed request.
6. Document the owner of each state value and its lifetime.

PROJECT 12 - Multi-screen planner with reliable state

Constraints, verification and report

Constraints

- No Redux or equivalent: justify ownership before adding a state tool.
- No disabling lifecycle checks to hide duplicate effect behavior.
- No fixed sleeps in E2E tests: wait for observable UI conditions.

Break it on purpose

- Delay goal A's response and return goal B first after switching selection. Observe A overwriting B on the broken branch; add stale-response handling and prove B remains selected.

Definition of done

- [] Shared and local state have explicit owners.
- [] The race and effect-cleanup checks pass.
- [] Three E2E journeys pass reliably.
- [] Focus and prior checks remain correct.

Stretch goals

- Optional: preserve a filter in the URL after explaining its ownership.

Concepts to read up on

- useEffect cleanup stale closure
- context reducer state ownership
- E2E locators auto-waiting request interception

Report back with

- Reducer/context, data-loading effect, and E2E files.
- Race reproduction and repaired recording.
- State-ownership notes.
- Explain why effect cleanup is necessary even if a component usually stays mounted.

PROJECT 13 - Defend public inputs

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 12.

Why now

Data that looks harmless in a form can become dangerous in a query or rendered page. You will protect three concrete boundaries and prove the defenses with adversarial inputs.

New topics

- Backend > Auth and security: XSS prevention.
- Backend > Auth and security: injection prevention.
- Backend > Auth and security: CORS and browser origin boundaries.

Reinforced topics

- P1 validation: specify length, type, and allowed fields.
- P8 SQL: parameterize user-controlled values.
- P11 React: render untrusted content through safe defaults.
- P7 integration tests: verify hostile inputs and origin behavior.

The build

Learners can save arbitrary plain-text notes without turning those notes into executable page content or SQL.

Requirements

1. Define maximum note length and accepted fields; reject unknown fields and invalid shapes consistently.
2. Store and display a note containing HTML-like text as literal text without executing it.
3. Keep all user-controlled SQL values parameterized; demonstrate a quote-containing note round-trips correctly.
4. Configure an explicit development origin policy if using separate frontend/backend origins; verify allowed and disallowed browser-origin behavior.
5. Add regression cases for script-like text, SQL-like text, oversized input, and unexpected properties.
6. Document why CORS is a browser policy and cannot replace authentication.

PROJECT 13 - Defend public inputs

Constraints, verification and report

Constraints

- No raw HTML rendering of notes.
- No SQL string concatenation with user values.
- No wildcard-with-credentials workaround or treating curl access as a CORS failure.

Break it on purpose

- On a disposable local branch, replace one safe text render with raw HTML insertion and use an inert local marker to demonstrate execution. Restore safe rendering and verify the regression test; never publish the intentionally vulnerable branch.

Definition of done

- [] Hostile strings remain data at both boundaries.
- [] Origin behavior matches the documented policy.
- [] Regression cases pass without weakening validation.
- [] The failure demonstration and repair are explained.

Stretch goals

- Optional: document one content-security policy defense without counting it as completed implementation.

Concepts to read up on

- stored reflected XSS output encoding
- SQL injection parameterized query
- same-origin policy CORS preflight

Report back with

- Validation, rendering, SQL, and CORS configuration files.
- Adversarial-input results and failure report.
- Explain why removing angle brackets is not a general XSS defense.

PROJECT 14 - Private accounts and permissions

Size: 3 week(s), 30-45 hours total | Mode: Extends Project 13.

Why now

Shared records need ownership before the app can be used by different people. Sessions and authorization add a clear boundary around every protected operation.

New topics

- Backend > Auth and security: server-side sessions and secure account handling.
- Backend > Auth and security: CSRF defenses for cookie-authenticated mutations.
- Backend > Auth and security: role-based access control.

Reinforced topics

- P13 security boundaries: reject unauthorized operations server-side.
- P8 SQL: model users, ownership, and session records.
- P6 HTTP: distinguish unauthenticated and forbidden requests.
- P12 E2E: test complete login and logout journeys.

The build

Two learners have private study logs; an administrator can manage a small public resource list.

Requirements

1. Implement registration/login/logout with a maintained session/auth library and maintained password-hashing implementation; document chosen settings.
2. Store server-side sessions in PostgreSQL; use appropriate HttpOnly, SameSite, expiry, and Secure cookie behavior for the execution environment.
3. Require ownership checks on all session reads/writes; implement learner and administrator roles for one resource-management action.
4. Protect cookie-authenticated mutations against CSRF with a documented library-supported mechanism.
5. Rotate the session identifier at login and invalidate it at logout; verify old cookies stop working.
6. Test two users, both roles, direct unauthorized API calls, missing/invalid CSRF proof, and the login/logout UI.

PROJECT 14 - Private accounts and permissions

Constraints, verification and report

Constraints

- No custom cryptography, plaintext passwords, or tokens in browser local storage.
- No client-only authorization or hidden-button security.
- No password reset, email delivery, or social login yet: keep account scope bounded.

Break it on purpose

- In a local test fixture, remove an ownership predicate and show user A reading B's record. Restore the check and prove the same direct API request is denied without relying on the UI.

Definition of done

- [] Cross-user access and role escalation are blocked.
- [] Login rotation, logout invalidation, and CSRF tests pass.
- [] Cookie settings and password handling are documented.
- [] Both user journeys pass through the real API and database.

Stretch goals

- Optional: add a session-list-and-revoke screen for the current user.

Concepts to read up on

- server-side sessions cookie flags
- password hashing maintained library
- CSRF token SameSite origin checking
- RBAC object ownership

Report back with

- Auth/session setup, ownership checks, schema, and tests.
- Two-user access matrix and cookie evidence with values redacted.
- Failure report.
- Explain why a valid login does not authorize access to every record.

PROJECT 15 - Sign in through an identity provider

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 14.

Why now

Many applications delegate identity, but delegation still needs verification. You will connect one provider while retaining the application's own sessions and permissions.

New topics

- Backend > Auth and security: OAuth/OIDC authorization-code flow with PKCE.
- Backend > Auth and security: JWT signature and claim validation through a maintained library.
- Backend > Auth and security: secrets handling and rotation.

Reinforced topics

- P14 sessions: convert verified provider identity into an application session.
- P14 RBAC: keep local permissions independent of provider login.
- P13 validation: reject malformed or incorrectly scoped identity results.

The build

A learner can sign in through one OIDC provider and return to the same private dashboard.

Requirements

1. Choose one provider when starting; use a local test provider if an external account is unsuitable. Document issuer and redirect configuration.
2. Use a maintained OIDC client and authorization-code flow with PKCE, state, and nonce validation where applicable.
3. Verify signature, issuer, audience, expiry, and expected token use; distinguish an ID token from an API access token.
4. Map the verified issuer-and-subject identity to a local user; do not auto-link accounts solely by an unverified email.
5. Load client secrets from environment/configuration outside Git, provide a placeholder-only example, and demonstrate replacing a test secret.
6. Test success, cancelled consent, wrong state, expired/wrong-audience test token, and provider unavailability.

PROJECT 15 - Sign in through an identity provider

Constraints, verification and report

Constraints

- No hand-written JWT verification or custom OAuth flow.
- No logging tokens, authorization codes, or client secrets.
- No replacing local authorization with 'provider says logged in'.

Break it on purpose

- Use a wrong audience in a disposable test fixture. Observe rejection despite a valid signature, restore the expected audience, and verify login. Rotate a test secret and show the old one no longer works.

Definition of done

- [] Provider login establishes a valid local session.
- [] Invalid claims and flow state are rejected.
- [] Secrets are absent from repository and logs.
- [] Existing local-login and ownership checks remain green.

Stretch goals

- Optional: support explicit linking while already authenticated, with reauthentication.

Concepts to read up on

- OAuth OIDC authorization code PKCE state nonce
- JWT issuer audience expiry ID token access token
- secret rotation environment configuration

Report back with

- Redacted provider/client configuration, identity mapping, and negative tests.
- Successful flow diagram, invalid-token evidence, and rotation report.
- Explain what JWT decoding does not verify.

PROJECT 16 - Smooth, accessible dashboard

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 15.

Why now

A working dashboard can still waste renders and exclude users. Measuring one slow interaction teaches performance decisions without turning optimization into guesswork.

New topics

- Frontend > React: reconciliation and the Fiber scheduling model.
- Frontend > React: measured render optimization.
- Common > Testing and performance: Web Vitals and Lighthouse interpretation.

Reinforced topics

- P12 state management: reduce unnecessary shared updates.
- P3 accessibility: audit the real authenticated workflow.
- P3 CSS: prevent avoidable layout shift.
- P9 profiling: compare a fixed before/after workload.

The build

A learner filters a 1,000-row synthetic session history without losing keyboard usability or seeing unstable layout.

Requirements

1. Create a reproducible slow filter interaction over synthetic data and record a React profiler baseline.
2. Explain component identity, render work, and commit work using the recorded interaction.
3. Make one measured optimization; show why each memoization or state move is necessary.
4. Run Lighthouse on the same route three times before and after under recorded conditions; report median lab results and distinguish them from field Web Vitals.
5. Check labels, focus order, error announcements, contrast, and keyboard operation manually; repair at least one identified issue.
6. Record one explicit performance budget for this fixture and report whether it is met.

PROJECT 16 - Smooth, accessible dashboard

Constraints, verification and report

Constraints

- No blanket memoization or changing the dataset between comparisons.
- No claiming a Lighthouse score proves accessibility or real-user performance.
- No removing important functionality to improve a score.

Break it on purpose

- Pass a freshly created unstable prop into the slow subtree and observe repeated rendering. Restore stable ownership/props, then verify the profiler and keyboard journey.

Definition of done

- [] Before/after evidence uses comparable conditions.
- [] Optimization preserves correctness.
- [] Manual accessibility checks pass.
- [] The explanation separates lab measurements, field metrics, render, and commit.

Stretch goals

- Optional: compare deferred rendering with the simpler optimized filter.

Concepts to read up on

- React reconciliation Fiber render commit profiler
- memo useMemo useCallback referential identity
- LCP INP CLS Lighthouse lab versus field

Report back with

- Profiler captures, changed components, and median Lighthouse results.
- Accessibility checklist and failure report.
- Explain one optimization you chose not to apply.

PROJECT 17 - Shareable server-rendered pages

Size: 3 week(s), 30-45 hours total | Mode: Extends Project 16; migrate only the public page surface first.

Why now

Client rendering delays useful content until JavaScript executes. A public goal page lets you compare server rendering while preserving private application behavior.

New topics

- Frontend > Rendering strategies: Next.js routing and server/client boundaries.
- Frontend > Rendering strategies: server-side rendering.
- Frontend > Rendering strategies: hydration.

Reinforced topics

- P11 React: keep interactive components small and explicit.
- P4 HTTP: inspect response HTML and statuses.
- P10 TypeScript: share safe public data shapes.
- P14 ownership: expose only explicitly public data.

The build

A learner marks a goal public and shares a URL whose useful content appears in the initial HTML.

Requirements

1. Add an explicit public flag and a public read contract; private goals must not become discoverable through the new route.
2. Create one Next.js public goal route using request-time rendering and one small interactive calculator widget.
3. Prove the initial HTML contains the public title and summary with JavaScript disabled.
4. Keep secrets and database/server-only modules out of client bundles; document the server/client boundary.
5. Return appropriate missing/private responses and retain the existing authenticated dashboard.
6. Add checks for a public goal, private goal, missing goal, and hydration without console warnings.

PROJECT 17 - Shareable server-rendered pages

Constraints, verification and report

Constraints

- No marking the entire app as a client component.
- No copying private API payloads wholesale into public page props.
- No full application rewrite: one public route and one interactive widget.

Break it on purpose

- Render a nondeterministic value differently on server and client in the local widget. Observe the hydration warning, make initial output deterministic, and verify both source HTML and the interactive page.

Definition of done

- [] Public content is present before client JavaScript.
- [] Private data remains private.
- [] No hydration mismatch remains.
- [] Existing dashboard and auth checks pass.

Stretch goals

- Optional: add public metadata with a meaningful title and description.

Concepts to read up on

- Next.js App Router server client components
- SSR response HTML hydration mismatch
- server-only imports public data projection

Report back with

- Route, public-data contract, and widget files.
- JavaScript-disabled evidence, response HTML, and negative-access results.
- Hydration failure report.
- Explain which work runs on the server and which must run in the browser.

PROJECT 18 - Fast public course catalogue

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 17.

Why now

Not every public page needs fresh request-time work. A tiny catalogue makes build-time generation, controlled staleness, and progressive responses comparable.

New topics

- Frontend > Rendering strategies: static site generation.
- Frontend > Rendering strategies: incremental static regeneration.
- Frontend > Rendering strategies: streaming with a suspense boundary.

Reinforced topics

- P17 SSR: choose rendering based on freshness requirements.
- P17 hydration: keep client interactivity deterministic.
- P11 React: isolate a delayed subview.

The build

Visitors browse six public courses; the catalogue loads quickly while one slow recommendation section arrives separately.

Requirements

1. Create six seeded public course pages and document which output is generated at build time.
2. Implement one explicitly configured regeneration policy for a course summary; record the framework version and relevant settings.
3. Demonstrate a content change before and after regeneration, including any stale response during regeneration.
4. Add one deliberately delayed noncritical section behind a visible loading boundary and show useful content arriving earlier.
5. Compare SSG, ISR, SSR, and streaming in a decision table for these specific routes.
6. Keep all personalized data outside shared static output.

PROJECT 18 - Fast public course catalogue

Constraints, verification and report

Constraints

- No relying on undocumented framework caching defaults.
- No artificial delay in core content or production behavior after the demonstration.
- No redesign or external CMS: use a small seeded content source.

Break it on purpose

- Change a course without triggering the configured regeneration path. Observe stale content, identify the generation/cache boundary, trigger the intended update, and verify refreshed HTML.

Definition of done

- [] Generation and refresh behavior are evidenced.
- [] Streaming delivers useful early content.
- [] No private data enters shared output.
- [] Rendering choices and version-sensitive settings are documented.

Stretch goals

- Optional: compare timed regeneration with an explicit update trigger.

Concepts to read up on

- SSG ISR revalidation staleness
- React Suspense streaming HTML
- rendering strategy freshness tradeoffs

Report back with

- Route/rendering configuration, content fixture, and streaming boundary.
- Before/after HTML and timing evidence.
- Stale-content failure report.
- Explain why streaming and caching answer different questions.

PROJECT 19 - Fresh pages without repeated work

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 18.

Why now

Rendering and caching can interact in surprising ways. One public response and one private response are enough to learn freshness, validation, and safe cache boundaries.

New topics

- Backend > Caching: HTTP caching and conditional requests.
- Backend > Caching: CDN behavior and cache keys.
- Backend > Caching: invalidation strategies.

Reinforced topics

- P4 HTTP: inspect request/response headers and status codes.
- P18 rendering: distinguish regenerated output from cached delivery.
- P9 performance: measure origin requests, not just perceived speed.
- P14 privacy: prevent shared caching of personalized content.

The build

Public course summaries reuse cached responses while a learner's private progress always follows the documented privacy policy.

Requirements

1. Define separate cache policies for versioned static assets, public course data, and private progress responses.
2. Implement an ETag/If-None-Match path for one public API response and prove unchanged content yields a bodyless 304.
3. Use a configurable hosting CDN for the public static/catalogue artifact; record cache-key inputs and observable hit/miss or origin-request evidence.
4. Update one public course and demonstrate the chosen invalidation or expiry path across browser and edge caches.
5. Test with two authenticated users and verify private content cannot be served across accounts.
6. Record freshness limits and an origin-request count for a repeatable sequence of five reads and one update.

PROJECT 19 - Fresh pages without repeated work

Constraints, verification and report

Constraints

- No caching authenticated responses at a shared edge in this exercise.
- No random query strings as the permanent invalidation solution.
- No stacking undocumented framework and CDN defaults: specify the behavior you rely on.

Break it on purpose

- Omit the public-content invalidation step, observe stale data at the edge, identify which cache served it, then repair the update path and verify new content from a fresh client.

Definition of done

- [] Conditional request and cache-policy checks pass.
- [] Origin-request evidence shows the intended reuse.
- [] Update freshness and cross-user isolation are verified.
- [] The stale response is traced to a specific cache layer.

Stretch goals

- Optional: compare stale-while-revalidate with your current policy.

Concepts to read up on

- Cache-Control ETag If-None-Match 304
- CDN cache key Vary Age
- cache invalidation browser edge origin

Report back with

- Header/policy configuration and update/invalidation code.
- Request sequence with headers and origin counts.
- Two-user privacy results and stale-cache report.
- Explain why a browser refresh does not necessarily refresh an edge cache.

PROJECT 20 - Releases that check themselves

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 19.

Why now

A release should correspond to a known revision that passed meaningful checks. Automation and a small look inside Git connect source history to the artifact users receive.

New topics

- Common > VCS and hosting / Dev environment: GitHub Actions and CI/CD.
- Common > VCS and hosting: tagged releases and rollback.
- Common > VCS and hosting: Git blob, tree, commit, and reference relationships.

Reinforced topics

- P4 PR workflow: require reviewable changes and recorded checks.
- P7 tooling: install and run checks reproducibly.
- P12 E2E: protect one critical user journey.
- P2 documentation: publish accurate release instructions.

The build

Every change to StudyTrack is checked automatically, and a tagged public-site release can be traced and rolled back.

Requirements

1. Create a PR workflow for locked installation, type/lint checks, unit/integration tests, a production build, and one critical E2E journey.
2. Use isolated service/test data in CI and avoid exposing secrets to untrusted contributions.
3. Create a release workflow or documented approved step that deploys the already-checked artifact for the public site.
4. Tag one release and record its commit, artifact identifier, and short release notes.
5. Inspect one commit's tree and one file's blob using Git's object tools; explain what the release tag references.
6. Demonstrate restoring the prior release and checking its visible version marker.

PROJECT 20 - Releases that check themselves

Constraints, verification and report

Constraints

- No rebuilding an unrecorded revision after approval.
- No suppressing a failing check or granting broad workflow permissions to make deployment easier.
- No storing secrets in workflow files.

Break it on purpose

- Introduce a calculator regression on a branch. Observe the failed check and blocked release path, repair it, then intentionally deploy a harmless wrong version marker and roll back to the prior artifact.

Definition of done

- [] A real failing change is caught.
- [] A successful release maps to a commit and artifact.
- [] Rollback restores the known marker.
- [] Git-object explanation and release notes are accurate.

Stretch goals

- Optional: add a dependency cache and measure whether it improves CI time.

Concepts to read up on

- GitHub Actions permissions artifacts environments
- continuous integration continuous delivery
- Git blob tree commit tag ref
- release rollback provenance

Report back with

- Workflow files, failed/successful run links, tag, and release notes.
- Git object relationship notes and rollback evidence.
- Explain why a branch name alone is an unstable release identifier.

PROJECT 21 - Reproducible container release

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 20.

Why now

A passing local build still depends on an implicit machine setup. Containerizing the existing application makes runtime assumptions explicit and testable.

New topics

- Common > Dev environment and deployment: Docker images, containers, networks, and volumes.
- Common > Dev environment and deployment: reproducible builds with pinned inputs.

Reinforced topics

- P1/P6 Node runtime: distinguish application exit from HTTP failure.
- P20 CI/CD: build and identify the exact runnable artifact.
- P4 HTTP: verify service readiness from outside its process.

The build

A fresh checkout starts StudyTrack and its PostgreSQL dependency through a documented container setup.

Requirements

1. Create a multi-stage production image where useful, run as a non-root user, and exclude secrets and development data from the build context.
2. Pin base/dependency images to explicit immutable digests for the recorded build; commit the package lockfile.
3. Provide one compose-based local setup for app and database with a named data volume.
4. Separate configuration from image contents and expose a health endpoint that reports only appropriate public information.
5. Verify create-record, restart-app, and recreate-container behavior without losing database records.
6. Record image digest and demonstrate starting the same image on a clean local setup.

PROJECT 21 - Reproducible container release

Constraints, verification and report

Constraints

- No latest tags, embedded credentials, or development server as the production command.
- No baking database data into an image.
- No Kubernetes yet: understand a single container deployment first.

Break it on purpose

- Start the app with a wrong database hostname in a disposable configuration. Observe unhealthy startup and connection errors, repair the setting, then verify readiness and persisted records.

Definition of done

- [] Pinned build and clean startup are reproducible.
- [] App runs non-root and contains no secrets.
- [] Data survives container replacement.
- [] Wrong-configuration diagnosis and repair are documented.

Stretch goals

- Optional: compare image size before and after removing unnecessary build dependencies.

Concepts to read up on

- Docker image container layer multi-stage
- container DNS network volume
- image digest reproducibility health check

Report back with

- Container/build/compose files and sanitized configuration example.
- Image digest, clean-start output, and persistence evidence.
- Failure report.
- Explain what a volume preserves that a container writable layer may not.

PROJECT 22 - Deploy to a Linux server

Size: 3 week(s), 30-45 hours total | Mode: Extends Project 21.

Why now

Managed publication hides process ownership and traffic entry. Deploying one existing app on a Linux VM gives you a concrete operating environment to inspect.

New topics

- Common > Fundamentals: Linux users, permissions, processes, and SSH.
- Backend > Web servers: nginx reverse proxying and TLS termination.
- Common > Dev environment and deployment: basic VM, storage, firewall, and service configuration.

Reinforced topics

- P1 CLI: diagnose paths, environment, and process state on another machine.
- P21 Docker: run the recorded image with persistent data.
- P4 HTTP/HTTPS: trace traffic through the proxy.
- P20 releases: deploy and restore identifiable versions.

The build

StudyTrack runs behind HTTPS on a Linux VM with a repeatable deploy and restart procedure.

Requirements

1. Use a local Linux VM by default; choose a public VM only when hosting budget/account access is agreed at project start.
2. Create a non-root deployment account with SSH key access and appropriate file permissions.
3. Run the existing pinned app/database setup; configure restart behavior and persistent data explicitly.
4. Place nginx in front of the app. Use a locally trusted lab certificate for a local VM or a publicly trusted certificate for a public hostname; document renewal/replacement.
5. Expose only necessary ingress, keep the database private, and verify access through the proxy.
6. Reboot the VM, confirm recovery, and record a deploy/rollback runbook including ownership and configuration locations.
7. Back up and restore a disposable PostgreSQL copy with native tools, reconcile session counts/totals, and add the tested commands to the runbook.

PROJECT 22 - Deploy to a Linux server

Constraints, verification and report

Constraints

- No root application process, public database port, or disabling TLS verification to claim success.
- No managed app platform for this exercise: operate the proxy and process yourself.
- One VM only; high availability comes later.

Break it on purpose

- Stop the app container while nginx remains running. Observe the proxy's upstream failure, distinguish it from DNS/TLS failure, restart the app, and verify a healthy request plus preserved data.

Definition of done

- [] HTTPS access and certificate trust are verified.
- [] Reboot recovers service and data.
- [] SSH, permissions, and ingress match the documented setup.
- [] A proxy-to-app outage is diagnosed and repaired.

Stretch goals

- Optional: automate the documented deployment steps without hiding failure output.

Concepts to read up on

- SSH keys Linux permissions processes signals
- nginx reverse proxy TLS termination
- VM firewall persistent storage restart policy

Report back with

- Redacted proxy/deployment configuration and runbook.
- Reboot, HTTPS, and persistence evidence.
- Outage report.
- Explain which process receives the browser's TLS connection.

PROJECT 23 - Diagnose an unreachable or overloaded site

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 22.

Why now

Several failures look like 'the website is down' to a user. Layer-by-layer diagnosis and a second upstream teach you to identify the actual failing hop.

New topics

- Common > Fundamentals > Networking: TCP/UDP diagnostics and connection behavior.
- Backend > Web servers: load balancing across two app instances.
- Backend > Web servers: a small Apache reverse-proxy comparison.

Reinforced topics

- P4 DNS: distinguish wrong name resolution from application failure.
- P22 TLS: inspect certificate trust/name/expiry separately.
- P22 reverse proxying: identify upstream status and logs.
- P22 processes/CLI: inspect listeners and filter relevant output.

The build

The deployed tracker serves through two stateless app instances, with an operator's guide for locating connection failures.

Requirements

1. Run two instances against the existing shared database; return a safe instance marker from a diagnostic endpoint.
2. Configure nginx load balancing and prove both instances receive requests; document existing session storage behavior.
3. Demonstrate a TCP connection probe and a bounded local UDP echo exchange; explain why UDP results do not have HTTP status codes.
4. Use command-line text tools to filter one timestamp/instance from logs; record the exact input and result.
5. On a separate local port, reproduce one proxy route in Apache and compare the relevant configuration and logs.
6. Diagnose a wrong hostname, a stopped upstream, and a deliberately invalid lab certificate using distinct evidence.

PROJECT 23 - Diagnose an unreachable or overloaded site

Constraints, verification and report

Constraints

- No random restart-until-fixed procedure: collect evidence before changing state.
- No claiming two processes equal complete high availability.
- No copying production credentials into the Apache comparison.

Break it on purpose

- Stop one upstream under a bounded read workload. Observe per-instance traffic and failed requests, repair failover/recovery configuration as needed, and prove service recovery. Also identify the separate DNS and certificate failures.

Definition of done

- [] Two-instance routing and shared-session behavior work.
- [] TCP/UDP and text-processing evidence is correct.
- [] Three failure classes are distinguished.
- [] nginx and Apache comparison includes a tested request.

Stretch goals

- Optional: measure behavior when one upstream is slow rather than stopped.

Concepts to read up on

- TCP handshake ports UDP datagram
- DNS versus TLS versus HTTP failure
- nginx upstream Apache ProxyPass
- ps lsof ss grep awk tail

Report back with

- Proxy configurations, instance evidence, and diagnostic notes.
- TCP/UDP transcript and filtered log example.
- Failure reports.
- Explain which layer a 502 implicates and what it does not prove.

PROJECT 24 - Explain production failures

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 23.

Why now

A healthy page does not explain an intermittent error. Correlated logs and one error-reporting integration make failures traceable to a request and release.

New topics

- Backend > Observability: structured logging.
- Backend > Observability: Sentry error reporting and release context.
- Common > Documentation: architecture decision records.

Reinforced topics

- P6 HTTP: correlate one request through handler and database work.
- P7 integration tests: check error behavior and sensitive-data handling.
- P20 releases: identify the failing revision.
- P22 deployment: document an operator's recovery action.

The build

An operator can investigate a failed study-session save using a request ID and release identifier.

Requirements

1. Emit structured logs with timestamp, level, request ID, route template, status, duration, and release; avoid raw request bodies and credentials.
2. Return or display a support-safe request ID for an unexpected failure.
3. Capture one synthetic backend error in Sentry with environment/release context and scrub sensitive fields.
4. Prove logs and the error event identify the same failed request.
5. Write two short ADRs: one for session storage and one for logging/redaction choices.
6. Document a five-step investigation path from user-visible error to verified repair.

PROJECT 24 - Explain production failures

Constraints, verification and report

Constraints

- No 'log everything' debugging or real private data in error events.
- No swallowing exceptions and returning success.
- No installing dashboards unrelated to the chosen failure.

Break it on purpose

- Force a database constraint failure on synthetic input. Observe the safe user error, correlated log, and Sentry event; fix the cause and verify a successful save without sensitive data leakage.

Definition of done

- [] One failure is correlated end to end.
- [] Redaction is verified with a sentinel secret in a test fixture.
- [] ADRs state context, decision, and consequences.
- [] The investigation runbook reproduces the diagnosis.

Stretch goals

- Optional: capture the corresponding frontend error with the same support-safe correlation context.

Concepts to read up on

- structured logs correlation ID route template
- Sentry release environment scrubbing
- ADR context decision consequences

Report back with

- Logger/error-reporting setup, redaction test, and two ADRs.
- Sanitized correlated log/event evidence.
- Failure report and investigation runbook.
- Explain why recording the release improves an error report.

PROJECT 25 - First portfolio release and review

Size: 3 week(s), 30-45 hours total | Mode: Extends Project 24.

Why now

Many completed exercises do not automatically become a coherent portfolio. This checkpoint tests whether you can ship, explain, and repair the application as one product.

New topics

- None. This is an integration, repair, or independent transfer assessment.

Reinforced topics

- P13-P15 security: demonstrate accounts, ownership, and hostile-input defenses together.
- P7/P12/P16 testing: verify core journeys and accessibility under one release.
- P20-P24 deployment/operations: release, diagnose, and roll back independently.
- P2/P7/P24 documentation: make setup, API use, and decisions understandable.

The build

A focused StudyTrack release that another person can evaluate without a guided tour.

Requirements

1. Freeze scope to sign-in, goal management, session recording, progress reports, and a public share page; disable incomplete experiments.
2. Prepare synthetic demo data and a repeatable fresh-checkout setup. Supply a public demo if hosting is available, otherwise a reproducible local deployment and uncut demonstration.
3. Pass type/lint, unit, integration, critical E2E, and manual keyboard checks; report performance conditions and known limitations.
4. Write a concise README, API guide, architecture diagram, and two honest case studies about failures you fixed.
5. Demonstrate a fresh release and rollback, including a database backup/restore rehearsal using a disposable copy.
6. Complete a 45-minute explanation/debugging review: trace one request, read one SQL plan, inspect one Git change, and repair a seeded defect without a complete solution.

PROJECT 25 - First portfolio release and review

Constraints, verification and report

Constraints

- No new features, tutorial scripts, fabricated users/scale, or unmeasured performance claims.
- No declaring readiness solely because checks are green.
- No moving on from an unexplained data-loss or authorization defect.

Break it on purpose

- Have the mentor select one previously studied defect category without revealing the location. Record hypothesis, evidence, repair, and regression check; separately restore a disposable backup and reconcile record totals.

Definition of done

- [] A reviewer can run the release from the instructions.
- [] Core security and user journeys pass.
- [] Deployment, rollback, and restore have evidence.
- [] The independent review produces an explicit readiness assessment and remaining gaps.

Stretch goals

- Optional: turn one case study into a short portfolio article.

Concepts to read up on

- technical demo narrative
- architecture explanation request lifecycle
- debugging hypothesis evidence
- junior full-stack interview project discussion

Report back with

- Release/repository/demo links or reproducible local package.
- Check results, README/API guide, case studies, and architecture diagram.
- Debugging and restore evidence.
- State what you can now build independently and which decisions you still cannot defend.

PROJECT 26 - Shared sessions and fair API limits

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 25.

Why now

Repeated reads and shared runtime state create different storage needs. Redis becomes useful when you can state which data it owns and which data can be rebuilt.

New topics

- Backend > Databases: Redis as a data store for expiring sessions.
- Backend > Caching: Redis cache-aside for one public summary.
- Backend > APIs: rate limiting.

Reinforced topics

- P14 sessions: preserve invalidation and ownership across two instances.
- P8 persistence: distinguish authoritative database data from derived cache entries.
- P19 HTTP caching: keep browser/edge/Redis policies separate.

The build

Two app instances share expiring login sessions, reuse a public summary cache, and enforce one consistent API limit.

Requirements

1. Move server-side sessions to Redis through the maintained session adapter; define the acceptable consequence of session loss and verify logout across instances.
2. Cache only one public aggregate using a documented key, TTL, and update invalidation rule; PostgreSQL remains authoritative.
3. Implement one atomic fixed-window rate limit using a supported Redis atomic mechanism; state its boundary-burst limitation.
4. Return 429 and a meaningful retry hint when the test limit is exceeded.
5. Test alternation between two app instances, cache hit/miss behavior, mutation invalidation, and expiry.
6. Document separate failure policies for authentication state, optional cache reads, and rate-limit state.

PROJECT 26 - Shared sessions and fair API limits

Constraints, verification and report

Constraints

- No caching private responses under shared keys.
- No non-atomic read-then-write counter.
- No pretending all Redis data is disposable merely because some keys are caches.

Break it on purpose

- Delete a synthetic session key and observe the expected reauthentication. Delete a public cache key and observe reconstruction without data loss; explain why the two outcomes differ.

Definition of done

- [] Cross-instance sessions and logout work.
- [] Aggregate caching stays correct after writes.
- [] Rate limit is consistent under concurrent requests.
- [] Data ownership and failure policies are explicit.

Stretch goals

- Optional: compare a sliding-window limiter against the documented fixed-window boundary.

Concepts to read up on

- Redis TTL session store cache-aside
- atomic increment transaction Lua script
- rate limit 429 Retry-After
- authoritative versus derived data

Report back with

- Redis key/policy table, adapter/cache/limiter files, and tests.
- Two-instance and concurrent-limit evidence.
- Failure report.
- Explain what must happen if Redis loses every session key.

PROJECT 27 - Survive a stale or unavailable cache

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 26.

Why now

Cache speed is not useful if cache failure makes the whole product fail unexpectedly. Metrics make the effect of fallback and recovery measurable.

New topics

- Backend > Observability: counters, gauges, and histograms.
- Backend > Observability: Prometheus collection and queries.
- Backend > Observability: Grafana dashboards.

Reinforced topics

- P26 Redis: apply different outage policies to sessions, cache, and limits.
- P19 invalidation: detect and repair stale public data.
- P9 load tests: compare throughput, latency, and errors under fixed conditions.

The build

An operator can see cache health and users receive controlled behavior when Redis is slow, stale, or unavailable.

Requirements

1. Instrument request rate/errors/duration, cache hits/misses, and Redis failures with bounded labels.
2. Run Prometheus and Grafana locally with one dashboard showing the selected route and cache behavior.
3. Set explicit Redis operation timeouts; apply the P26 failure policies without bypassing authentication.
4. Run the same five-client workload with warm cache, empty cache, and Redis stopped; record latency/error differences and database impact.
5. Demonstrate a stale cached total after a mutation on a broken branch and verify the repaired invalidation path.
6. Prevent simultaneous cache misses from producing unbounded duplicate expensive work within one app process; state the limitation across instances.

PROJECT 27 - Survive a stale or unavailable cache

Constraints, verification and report

Constraints

- No user IDs, raw URLs, or request IDs as metric labels.
- No silent fail-open authentication.
- No claiming a cache improves reliability without an outage measurement.

Break it on purpose

- Stop Redis during the workload. Observe timeout/error and database metrics, verify controlled user behavior, restart Redis, and confirm recovery without a restart loop or stale aggregate.

Definition of done

- [] Metrics and dashboard answer the stated operational questions.
- [] Three comparable workload runs are recorded.
- [] Outage behavior matches each data class's policy.
- [] Staleness and recovery are verified.

Stretch goals

- Optional: measure a bounded randomized TTL to reduce synchronized expiry.

Concepts to read up on

- Prometheus counter gauge histogram labels
- Grafana query dashboard
- cache stampede timeout fallback
- metric cardinality

Report back with

- Instrumentation, Prometheus/Grafana configuration, and dashboard export.
- Three workload results and screenshots.
- Outage/staleness report.
- Explain why a request ID belongs in a log or trace rather than a metric label.

PROJECT 28 - Generate reports in the background

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 27.

Why now

A slow report should not keep an HTTP request open indefinitely. A worker separates accepting work from completing it and makes retries an explicit product behavior.

New topics

- Backend > Async workflows: a durable job queue and worker.
- Backend > Async workflows: bounded retries with backoff.
- Backend > Async workflows: dead-letter handling.

Reinforced topics

- P8 persistence: store job ownership, status, and output references.
- P6 promises: separate enqueue acknowledgment from completion.
- P24 structured logs: correlate a request and its job.
- P27 metrics: show job backlog and failures.

The build

A learner requests a CSV progress report, watches its status, and downloads it when ready.

Requirements

1. Use one maintained Redis-backed queue library and one worker process; retain PostgreSQL metadata for user-visible job status.
2. Implement request, status, and authorized-download endpoints with pending/running/succeeded/failed states.
3. Generate reports from synthetic existing sessions and store output in a documented persistent local location.
4. Configure a maximum attempt count and backoff; classify one transient and one permanent failure.
5. Provide an inspect-and-retry procedure for dead jobs with an audit entry.
6. Verify that only the requesting learner can view or download the result.

PROJECT 28 - Generate reports in the background

Constraints, verification and report

Constraints

- No Kafka yet and no separate service per job type.
- No infinite retries or silently dropped failed jobs.
- No assuming one delivery means one execution: use a deterministic job/output identity.

Break it on purpose

- Kill the worker after output creation but before acknowledgment. Observe redelivery, ensure the same job does not expose duplicate outputs, then force a permanent failure and inspect its dead-letter state.

Definition of done

- [] Report request-to-download journey works.
- [] Worker restart does not lose accepted work.
- [] Retries stop and dead jobs remain inspectable.
- [] Ownership and duplicate-execution checks pass.

Stretch goals

- Optional: add cancellation only for jobs that have not started.

Concepts to read up on

- durable queue producer worker acknowledgment
- at-least-once execution retry backoff jitter
- dead-letter job state machine

Report back with

- Queue/worker setup, job schema, API/UI, and tests.
- Successful report, retry history, and dead-job evidence.
- Crash report.
- Explain why a queue acknowledgment is different from an HTTP acknowledgment.

PROJECT 29 - Reliable webhook delivery

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 28.

Why now

An external recipient may process a request even when you never receive its response. Webhooks make duplicate-safe effects and uncertain outcomes unavoidable.

New topics

- Backend > APIs: webhooks with authenticated delivery.
- Backend > APIs: idempotency keys and effect deduplication.

Reinforced topics

- P28 queues: deliver notifications through bounded retries.
- P8 transactions: persist deduplication and effect atomically.
- P6 REST: make retry-related responses explicit.
- P7 integration tests: exercise a real local receiver.

The build

When a report finishes, StudyTrack notifies a local partner receiver without applying the same partner-side effect twice.

Requirements

1. Create one allowlisted local receiver fixture and emit a report-ready event with a stable event ID and documented payload.
2. Sign deliveries using a maintained cryptographic API and a configured secret; verify signature and a replay-time policy at the receiver.
3. Store the event ID and receiver-side effect in one transaction so a duplicate returns the same successful outcome.
4. Add an idempotency key to report-request creation; the same key and payload return the same job, while conflicting payload reuse is rejected.
5. Record delivery attempt, status, and next retry time without logging secrets.
6. Test normal delivery, duplicate delivery, invalid signature, receiver failure, and a processed request whose response is lost.

PROJECT 29 - Reliable webhook delivery

Constraints, verification and report

Constraints

- No arbitrary user-supplied destinations: avoid expanding into URL-fetch security and subscription management.
- No deduplication held only in process memory.
- No claim of exactly-once network delivery: prove once-only business effects within the defined boundary.

Break it on purpose

- Make the receiver commit its effect and then drop the connection before responding. Observe the sender retry, verify only one effect exists, and restore normal responses.

Definition of done

- [] Duplicate event and duplicate request effects are prevented.
- [] Signature and replay checks pass.
- [] Retries are bounded and inspectable.
- [] Uncertain delivery outcome is demonstrated and explained.

Stretch goals

- Optional: replay a dead delivery safely through the same deduplication path.

Concepts to read up on

- webhook HMAC signature timestamp replay
- idempotency key payload conflict
- atomic deduplication effect
- delivery versus processing guarantee

Report back with

- Payload contract, sender, receiver, deduplication schema, and tests.
- Lost-response timeline and effect counts.
- Explain where the idempotency guarantee begins and ends.

PROJECT 30 - Live study-room activity

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 29.

Why now

Polling every screen treats all updates the same way. Comparing one-way events with a two-way room interaction clarifies which connection model the product needs.

New topics

- Backend > APIs: server-sent events.
- Backend > APIs: WebSockets.

Reinforced topics

- P6 HTTP/async JavaScript: manage long-lived connections.
- P14 sessions: authorize connection establishment and room access.
- P29 event identity: recognize duplicate updates.
- P12 lifecycle: clean up subscriptions when views change.

The build

A learner sees report-completion events live and can send a simple ready/not-ready signal in a two-person study room.

Requirements

1. Use SSE for report-status notifications and WebSockets for the small two-way room signal.
2. Authenticate both connection paths and enforce room membership server-side; check browser origins appropriately.
3. Provide visible connected/reconnecting/offline states and bounded reconnection delays.
4. Give events stable IDs and implement a bounded recent-event replay or an explicit resynchronization fetch after reconnect.
5. Close listeners and sockets on navigation/logout; document how server-side session invalidation affects existing connections.
6. Test with two browser contexts and keep database state authoritative for completed reports.

PROJECT 30 - Live study-room activity

Constraints, verification and report

Constraints

- No hosted realtime service or full chat platform.
- No trusting a room ID or user ID sent by the client.
- No claiming a reconnected socket implies no events were missed.

Break it on purpose

- Disconnect one client while an event occurs, then reconnect it. Observe the gap, replay or resynchronize, and verify one correct visible update without duplicated listeners.

Definition of done

- [] SSE and WebSocket features serve distinct needs.
- [] Unauthorized room access fails.
- [] Disconnect, reconnect, logout, and cleanup are tested.
- [] The missed-event policy is explicit and works.

Stretch goals

- Optional: add a small connection-count metric with bounded labels.

Concepts to read up on

- SSE EventSource Last-Event-ID
- WebSocket upgrade connection lifecycle
- reconnect resynchronization session invalidation

Report back with

- Both connection handlers, client hooks, and access tests.
- Two-client recording and reconnection report.
- Explain why the report stream does not need a bidirectional protocol.

PROJECT 31 - Flexible dashboard data and access

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 30.

Why now

A dashboard may need a composed response and rules richer than fixed roles. One narrow query lets you compare an interface-specific backend and attribute-based access.

New topics

- Backend > APIs: GraphQL schema, query, and resolver contracts.
- Backend > APIs: backend-for-frontend composition.
- Backend > Auth and security: attribute-based access control.

Reinforced topics

- P6 REST: reuse underlying service functions and compare request shapes.
- P14 authorization: keep checks at the data boundary.
- P10 TypeScript: model nullable and missing results.
- P9 SQL: avoid multiplying queries per row.

The build

A dashboard requests a learner's goal, recent sessions, and report status in one query; a mentor may view only assigned learners during an active assignment.

Requirements

1. Implement one read-only GraphQL query with a small bounded result; keep writes on the existing REST API.
2. Place dashboard composition in an explicit BFF module that calls existing application functions rather than duplicating persistence.
3. Add an assignment record with mentor, learner, active period, and organization; enforce those attributes for one read policy.
4. Verify owner, assigned mentor, expired assignment, different organization, and anonymous cases.
5. Set query depth/size limits appropriate to the small schema and inspect database query count for the dashboard.
6. Document REST versus GraphQL error/null behavior for this query.

PROJECT 31 - Flexible dashboard data and access

Constraints, verification and report

Constraints

- No generic query-everything schema, GraphQL mutations, or new service deployment.
- No authorization solely at the top-level resolver if nested data needs its own boundary.
- No fetching all records and filtering unauthorized data in the browser.

Break it on purpose

- Expire a mentor assignment while retaining the same role and session. Observe that role-only logic would allow access, enforce the attribute policy, and verify the next read is denied.

Definition of done

- [] One dashboard query works with bounded database work.
- [] The five-case access matrix passes.
- [] BFF and persistence responsibilities remain separate.
- [] Attribute change affects authorization correctly.

Stretch goals

- Optional: add a query batching helper only if measurement shows an N+1 problem.

Concepts to read up on

- GraphQL schema resolver nullability query cost
- backend for frontend composition
- ABAC subject resource environment attributes

Report back with

- Schema/resolvers, BFF module, assignment model, and policy tests.
- Query count and access matrix.
- Expired-assignment report.
- Explain which decision belongs in the BFF and which belongs in authorization.

PROJECT 32 - Flexible learning-resource library

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 31.

Why now

Some resources have different fields that do not fit one convenient row shape. A small document store comparison teaches modelling tradeoffs without replacing the relational core.

New topics

- Backend > Databases: NoSQL document modelling and a MongoDB data store.

Reinforced topics

- P8 schema design: define required fields and relationships deliberately.
- P9 indexes: measure one resource query with and without an index.
- P7 API contracts: keep flexible storage behind validated input.
- P14 authorization: enforce ownership of resource edits.

The build

Learners save articles, videos, and books with a few type-specific fields and browse them by topic.

Requirements

1. Keep users/goals/sessions in PostgreSQL. Store only resources in a local MongoDB collection.
2. Support article, video, and book resource types with shared title/topic/owner fields and a small validated type-specific section.
3. Implement create, edit, and topic-filtered list through the existing UI/API; reject unknown resource types.
4. Seed 2,000 synthetic resources and compare a topic query plan before and after one index.
5. Define a schema version and migrate one older document fixture without losing its data.
6. Document why embedding or referencing was chosen for one relationship.

PROJECT 32 - Flexible learning-resource library

Constraints, verification and report

Constraints

- No schema-free interpretation of NoSQL: define and validate the document shape.
- No distributed transaction between PostgreSQL and MongoDB.
- No moving the whole product to a second database.

Break it on purpose

- Insert an old-version synthetic document that lacks a now-required field. Observe the failing consumer, migrate or adapt through the documented version path, and verify both old and new fixtures.

Definition of done

- [] Three resource types work with validated boundaries.
- [] Index evidence and schema migration are reproducible.
- [] Ownership checks remain intact.
- [] The relational/document ownership boundary is clear.

Stretch goals

- Optional: compare the same tiny resource model in PostgreSQL JSON storage in an ADR.

Concepts to read up on

- document database MongoDB schema validation
- embedding referencing index explain
- schema version migration

Report back with

- Resource schema/validation, API/UI, migration, and index definition.
- Query plans and old-document repair report.
- Explain one cost of introducing a second database.

PROJECT 33 - Search resources by intent

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 32.

Why now

Database filters do not provide useful full-text ranking by themselves. A bounded search feature introduces an index as a rebuildable projection of authoritative records.

New topics

- Backend > Search and analytics: Elasticsearch mapping and indexing.
- Backend > Search and analytics: relevance and analyzers.
- Backend > Search and analytics: Kibana exploration.

Reinforced topics

- P32 NoSQL: retain the resource collection as source of truth.
- P28 jobs: index changes outside the request path.
- P7 API contracts: expose a small search response.
- P9 measurement: evaluate a fixed fixture rather than impressions.

The build

Learners search the resource library by words in titles and descriptions and receive a useful ranked result list.

Requirements

1. Run one local Elasticsearch/Kibana pair and index a bounded public-resource projection; exclude private notes and secrets.
2. Define explicit mappings for title, description, topic, resource ID, and update version.
3. Add a search box with query, empty-result, and failure states; limit returned results.
4. Create ten fixed queries with expected relevant resources and measure a simple top-three relevance score.
5. Compare a title boost or analyzer choice using those queries; record why you retain the chosen setting.
6. Use Kibana to inspect one indexed document and one query/result set; document how to rebuild the index.

PROJECT 33 - Search resources by intent

Constraints, verification and report

Constraints

- No treating Elasticsearch as authoritative storage.
- No indexing arbitrary application payloads.
- No tuning against changing examples: freeze the relevance fixture first.

Break it on purpose

- Change a source resource without updating the search projection. Observe the stale result, compare source and indexed versions, reindex that resource, and verify corrected search output.

Definition of done

- [] Search works through the user interface.
- [] Mapping and relevance evaluation are reproducible.
- [] Kibana evidence matches the actual indexed data.
- [] Stale projection diagnosis and repair are recorded.

Stretch goals

- Optional: compare one synonym rule against the same relevance fixture.

Concepts to read up on

- Elasticsearch mapping analyzer inverted index
- relevance boost top-k evaluation
- Kibana Dev Tools Discover
- search index projection

Report back with

- Mapping, indexing job, search API/UI, and query fixture.
- Relevance comparison and Kibana evidence.
- Stale-index report.
- Explain why rebuilding a search index should not destroy the original resources.

PROJECT 34 - Repair stale search and build usage reports

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 33.

Why now

A search feature is incomplete if missed jobs leave it permanently wrong. Reconciliation and one small analytical view reuse the same data path at greater depth.

New topics

- None. This is an integration, repair, or independent transfer assessment.

Reinforced topics

- P32 document storage: reconcile authoritative IDs and versions.
- P33 Elasticsearch: rebuild projections and handle deletions.
- P33 Kibana: aggregate synthetic search usage by topic.
- P28 job recovery: retry failed indexing without duplicate effects.

The build

An operator repairs a stale search index and sees which public resource topics receive the most synthetic searches.

Requirements

1. Add a reconciliation command comparing a bounded source snapshot with index IDs/versions, including deleted resources.
2. Rebuild into a new versioned index, verify counts and the ten-query relevance fixture, then switch a read alias.
3. Capture only a synthetic query category, result count, and timestamp for the exercise; avoid identifying user-level analytics.
4. Build one Kibana view showing zero-result searches and top categories over a selected interval.
5. Demonstrate retrying a failed indexing job without regressing a newer resource version.
6. Write a source-to-index recovery runbook with a rollback step.

PROJECT 34 - Repair stale search and build usage reports

Constraints, verification and report

Constraints

- No manually editing search documents as the permanent repair.
- No overwriting newer projections with delayed older jobs.
- No broad analytics platform or tracking real people.

Break it on purpose

- Drop one indexing job and reorder two updates to the same resource. Observe a missing/stale result, run reconciliation, and verify the final index matches the latest source and relevance fixture.

Definition of done

- [] Missing, changed, and deleted records reconcile correctly.
- [] Alias rebuild and rollback work.
- [] Delayed events cannot overwrite newer versions.
- [] The analytics view reflects the synthetic fixture.

Stretch goals

- Optional: record rebuild time and estimate the limit of the current single-worker approach.

Concepts to read up on

- search reindex alias version reconciliation
- idempotent projection ordering
- Kibana aggregation zero-result rate

Report back with

- Reconciliation/rebuild tools, version checks, and dashboard export.
- Counts and relevance results before/after alias switch.
- Dropped/reordered-job report.
- Explain why equal record counts alone do not prove an index is correct.

PROJECT 35 - Follow a request across processes

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 34.

Why now

A request ID helps within logs, but timings across API and worker boundaries remain fragmented. Tracing makes one complete report workflow inspectable.

New topics

- Backend > Observability: distributed traces, spans, and context propagation.
- Backend > Observability: OpenTelemetry instrumentation and export.

Reinforced topics

- P24 structured logs: include trace correlation without exposing credentials.
- P27 metrics: connect aggregate latency to an individual example.
- P28 workers: carry or link execution context across the queue.
- P6 HTTP: distinguish server work from downstream waiting.

The build

An operator follows a report request from browser/API through queue, worker, database, and completion.

Requirements

1. Instrument one report workflow with OpenTelemetry and use one local collector/trace backend.
2. Show HTTP, database, enqueue, and worker spans with meaningful names and attributes.
3. Propagate context or use an appropriate span link across queued work; explain the choice rather than forcing a misleading parent duration.
4. Correlate structured logs with trace/span IDs; retain bounded metric labels.
5. Record one successful and one failed workflow, including error status and useful timing.
6. Configure a documented sampling policy and scrub test secrets from exported attributes.

PROJECT 35 - Follow a request across processes

Constraints, verification and report

Constraints

- No instrumenting every function or exporting request bodies.
- No claiming a trace is complete without verifying context across the worker boundary.
- No replacing existing metrics with trace screenshots.

Break it on purpose

- Remove queue context propagation on a temporary branch. Observe disconnected traces, restore propagation/linking, and prove the report's API and worker activity can be followed together.

Definition of done

- [] The chosen workflow is traceable across processes.
- [] Success/failure spans and log correlation are correct.
- [] Sampling and sensitive-data rules are documented.
- [] The deliberately broken propagation is diagnosed and repaired.

Stretch goals

- Optional: add one custom span around a measured expensive calculation.

Concepts to read up on

- OpenTelemetry tracer span context propagation
- traceparent span links asynchronous messaging
- collector exporter sampling

Report back with

- Instrumentation/collector configuration and sanitized trace exports.
- Successful, failed, and disconnected/repared traces.
- Explain why a slow HTTP span does not by itself identify the slow dependency.

PROJECT 36 - Alerts that identify actionable failures

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 35.

Why now

Charts do not tell an operator when action is required. A small service objective and tested alerts connect measurements to a concrete response.

New topics

- Backend > Observability: service-level indicators/objectives.
- Backend > Observability: alert rules, routing, and recovery.

Reinforced topics

- P35 traces: identify a slow dependency behind an alert.
- P27 metrics: choose correct windows and denominators.
- P24 Sentry: group errors by release during investigation.
- P9 load testing: generate repeatable good and bad traffic.

The build

An operator receives a local actionable alert when report creation becomes unreliable, and can resolve it using a short runbook.

Requirements

1. Choose one report-request success indicator and one latency indicator; define the window, denominator, target, and lab limitations.
2. Create two alerts with persistence windows and a documented low-traffic policy; route them to a local receiver so no external messaging account is required.
3. Each alert must contain symptom, affected route, dashboard link, and first investigation step.
4. Use a bounded workload to demonstrate healthy traffic, a brief tolerated blip, sustained failure, and recovery.
5. Correlate one firing alert with metrics, a trace, and the relevant Sentry release/error.
6. Write a runbook that states how to verify recovery and when to escalate.

PROJECT 36 - Alerts that identify actionable failures

Constraints, verification and report

Constraints

- No alert on every exception or every single slow request.
- No user IDs or raw paths in alert dimensions.
- No claiming a short lab run demonstrates a month-long availability objective.

Break it on purpose

- Add controlled latency or error responses to the report dependency. Observe the alert timing, diagnose with trace/log evidence, remove the fault, and verify the alert resolves after its documented window.

Definition of done

- [] Both alerts fire and recover as specified.
- [] Brief noise is treated according to policy.
- [] Runbooks lead to the measured cause.
- [] Targets are described as design objectives, not achieved historical uptime.

Stretch goals

- Optional: test what happens when telemetry itself stops arriving.

Concepts to read up on

- SLI SLO error budget
- Prometheus alert for window low traffic
- alert routing runbook recovery

Report back with

- Indicator definitions, alert rules, local receiver output, and runbooks.
- Healthy/blip/failure/recovery timeline.
- Correlated trace and sanitized error evidence.
- Explain which user impact each alert represents.

PROJECT 37 - Activity events across two services

Size: 3 week(s), 30-45 hours total | Mode: Extends Project 36; extract only the activity summary boundary.

Why now

A queue sends work to a worker, while an event log can support independent consumers. One extracted read service lets you learn that distinction without fragmenting the whole app.

New topics

- Backend > Async workflows: Kafka topics, partitions, offsets, and consumer groups.
- Backend > Async workflows: event-driven contracts.
- Backend > APIs: a small microservice boundary.

Reinforced topics

- P28 queues: compare a job command with a published fact.
- P29 idempotency: make replay safe.
- P8 schemas: give each service explicit data ownership.
- P35 tracing: correlate publication and consumption.

The build

A separate activity-summary service consumes session-recorded events and supplies a learner's summary to the existing dashboard.

Requirements

1. Run one local Kafka broker with persistent storage and document its single-broker limitations.
2. Define one versioned session-recorded event with stable event ID, learner/goal IDs, duration, timestamp, and aggregate key.
3. Extract only the summary reader/consumer into a separately runnable service with its own schema and database credentials; it must not query the core service's tables.
4. Partition consistently by the chosen aggregate and explain ordering scope.
5. Store processed event IDs or another durable deduplication mechanism; rebuild the summary from a controlled replay.
6. Show the existing UI reading the summary, including a visible freshness indicator; document the temporary database-write/publish gap that P38 will repair.

PROJECT 37 - Activity events across two services

Constraints, verification and report

Constraints

- No more than two business services or one Kafka topic in this assignment.
- No shared-table access across service boundaries.
- No assuming global event order or exactly-once business effects.

Break it on purpose

- Stop the consumer, create ten sessions, and observe stale summary/consumer lag. Restart it, verify catch-up and exact totals, then replay the same events and prove totals do not double.

Definition of done

- [] UI integrates the extracted service.
- [] Ownership, event shape, and ordering are explicit.
- [] Catch-up and duplicate replay preserve totals.
- [] The publication gap is documented rather than hidden.

Stretch goals

- Optional: add a second consumer group that only audits synthetic event counts.

Concepts to read up on

- Kafka topic partition offset consumer group lag
- event versus command
- microservice data ownership
- eventual consistency replay

Report back with

- Service boundary diagram, event contract, Kafka setup, and consumer tests.
- Lag/catch-up/replay evidence and UI freshness behavior.
- Explain why a broker acknowledgment does not make a database write and publication atomic.

PROJECT 38 - Recover missing and duplicate events

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 37.

Why now

P37 deliberately exposed the gap between committing data and publishing an event. An outbox closes that gap using a transaction you already understand.

New topics

- Backend > Async workflows: transactional outbox publishing.

Reinforced topics

- P8 transactions: commit the business row and outgoing event together.
- P37 Kafka: retry publication and replay consumption.
- P29 idempotency: deduplicate uncertain publication outcomes.
- P24 logging: reconstruct the crash sequence.

The build

Recorded study sessions eventually reach the activity service even if the app crashes at an inconvenient moment.

Requirements

1. Write the session and outbox event in one PostgreSQL transaction.
2. Implement one bounded publisher that retries unsent rows and marks progress only after the configured broker acknowledgment.
3. Keep event IDs stable across attempts and retain durable consumer deduplication.
4. Demonstrate crashes before database commit, after commit/before publish, and after publish/before outbox acknowledgment.
5. Add an outbox backlog metric and a documented stuck-row inspection procedure.
6. State the achieved delivery/effect guarantees and remaining single-publisher/broker limits.
7. Reuse the outbox pattern for the single P29 webhook event: preserve its delivery ID and verify receiver deduplication after a publisher restart.

PROJECT 38 - Recover missing and duplicate events

Constraints, verification and report

Constraints

- No distributed transaction spanning PostgreSQL and Kafka.
- No deleting an outbox row before publication is acknowledged.
- No changing event IDs on retries or claiming the network delivers once.

Break it on purpose

- Kill the publisher after Kafka accepts an event but before the outbox is marked sent. Observe republishing after restart, verify one summary effect, and reconcile sessions, outbox rows, and consumer totals.

Definition of done

- [] All three crash windows have defined outcomes.
- [] Committed sessions are eventually reflected after recovery.
- [] Duplicate publication does not double totals.
- [] Backlog and reconciliation evidence show completion.

Stretch goals

- Optional: run two publishers with a proven database claim/locking strategy.

Concepts to read up on

- transactional outbox atomic commit
- publisher acknowledgment uncertain outcome
- consumer idempotency reconciliation

Report back with

- Outbox migration, publisher, consumer deduplication, and crash tests.
- Three timelines plus record/event/summary reconciliation.
- Explain the difference between preventing loss and preventing duplicate delivery.

PROJECT 39 - One entrance for multiple services

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 38.

Why now

Clients should not need to know every internal service address. A gateway centralizes transport concerns while the BFF and services keep their existing responsibilities.

New topics

- Backend > APIs: API gateways and request routing.

Reinforced topics

- P31 BFF: compose dashboard data without moving business policy into the gateway.
- P37 microservices: preserve service-owned data and contracts.
- P26 rate limits: enforce a shared limit across upstream instances.
- P15 identity: validate the chosen external credential/session boundary.

The build

The browser uses one public API origin while requests reach the core and activity services through a controlled gateway.

Requirements

1. Route two explicit public paths to the existing services using a maintained gateway/proxy; configure upstream timeouts and request-size limits.
2. Propagate correlation context and forward only an allowlisted set of identity-related headers after validation.
3. Reject client attempts to spoof trusted internal identity headers.
4. Apply the shared rate limiter at one documented layer without unintentionally charging a request twice.
5. Keep service authorization in place and document which concerns belong to gateway, BFF, and service.
6. Make the dashboard show core data with an explicit unavailable/stale activity section when only the summary service fails.

PROJECT 39 - One entrance for multiple services

Constraints, verification and report

Constraints

- No business workflows, direct database access, or arbitrary proxy URLs in the gateway.
- No bypassing service authorization because traffic came from the gateway.
- No hidden automatic retries of non-idempotent writes.

Break it on purpose

- Stop the activity service. Observe the gateway timeout and partial dashboard failure, preserve core functions, restart the service, and verify recovery; test a spoofed identity header separately.

Definition of done

- [] One browser-facing origin serves both paths.
- [] Trust and rate-limit policies are verified.
- [] Upstream failure has bounded latency and clear UI behavior.
- [] Layer responsibilities and recovery evidence are documented.

Stretch goals

- Optional: compare two safe timeout settings under the same slow-upstream fixture.

Concepts to read up on

- API gateway routing timeout header trust
- BFF versus gateway
- retry safety idempotent method

Report back with

- Gateway configuration, trust tests, and responsibility diagram.
- Spoofed-header and partial-outage evidence.
- Explain why a gateway is not automatically an authorization boundary for every resource.

PROJECT 40 - Shared packages with clear contracts

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 39; reorganize the existing repository rather than starting over.

Why now

Multiple applications now share types and utilities. Explicit package boundaries prevent accidental coupling and give advanced types a concrete contract to protect.

New topics

- Frontend > Tooling: monorepos and package workspaces.
- Frontend > Tooling > TypeScript: mapped/conditional types for one shared contract.
- Common > Documentation: developer onboarding and package-boundary documentation.

Reinforced topics

- P2 modules: expose small public interfaces.
- P10 bundlers/transpilers: distinguish source, build output, and type declarations.
- P20 CI: verify affected applications from a clean install.
- P7 API contracts: keep runtime validation next to shared shapes.

The build

A contributor can find the web app, core API, activity service, and shared contracts and change one contract safely.

Requirements

1. Use one package manager's workspaces for web, core, activity, and shared-contract packages; retain one committed lockfile.
2. Define explicit package exports and forbid importing another package's internal files.
3. Create one mapped or conditional type that derives a useful request/result shape; show positive and negative type-check fixtures.
4. Keep runtime validation for network input and demonstrate a payload that is type-shaped in development but invalid at runtime.
5. Provide a root build/check command and document dependency order, package ownership, and local startup.
6. Verify a clean install/build and one deliberate cross-package contract change.

PROJECT 40 - Shared packages with clear contracts

Constraints, verification and report

Constraints

- No adding a separate monorepo orchestration platform unless ordinary workspace scripts demonstrably fail.
- No clever recursive types without a real consumer.
- No weakening boundaries with path aliases that bypass package exports.

Break it on purpose

- Remove a required exported field in the shared contract. Observe both affected consumers fail clearly, update them intentionally, and verify an unaffected package still builds.

Definition of done

- [] One clean install reproduces all relevant builds.
- [] Package boundaries and dependency graph are documented.
- [] Type-level fixtures catch the intended misuse.
- [] Runtime validation remains intact.

Stretch goals

- Optional: measure a build cache on the same clean/unchanged workload.

Concepts to read up on

- npm workspaces package exports dependency graph
- mapped conditional types type tests
- declaration output runtime validation

Report back with

- Workspace/package configuration, shared types, and developer guide.
- Dependency graph and cross-package failure output.
- Explain what shared types can and cannot guarantee across independently deployed services.

PROJECT 41 - Find a blocked event loop and memory leak

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 40.

Why now

An application can slow down without a slow database. Runtime profiling lets you distinguish CPU blocking from retained memory and choose a targeted repair.

New topics

- Frontend > JS / TS / Node: event-loop scheduling internals.
- Frontend > JS / TS / Node: memory, reachability, and garbage collection.

Reinforced topics

- P6 concurrency: distinguish asynchronous waiting from CPU work.
- P9 profiling: use fixed synthetic inputs.
- P27 metrics: relate event-loop delay and memory to request latency.
- P30 connections: inspect listener and connection lifetimes.

The build

The tracker keeps ordinary requests responsive while a synthetic report calculation runs, and repeated room visits stop retaining unused objects.

Requirements

1. Create a bounded local CPU-heavy report fixture and record request latency plus event-loop delay while it runs.
2. Explain the observed timer/promise/I/O scheduling using a small documented experiment; do not rely on an unspecified universal ordering.
3. Move or partition the heavy work using the already available job/process boundary, then compare ordinary-request latency.
4. Induce a retained-listener or connection-reference leak in a disposable fixture and collect before/after heap evidence.
5. Repair ownership/cleanup and compare retained object count after the same 100 open/close cycles.
6. Record memory and latency measurements with the same workload and runtime version.

PROJECT 41 - Find a blocked event loop and memory leak

Constraints, verification and report

Constraints

- No fixing CPU blocking by merely adding async to a function.
- No increasing memory limits as the only leak repair.
- No forcing garbage collection as a production cleanup strategy.

Break it on purpose

- Run the CPU block and leak fixtures separately so their symptoms are distinguishable. Use profiling evidence to locate each cause, repair it, and repeat the identical workload.

Definition of done

- [] Two different runtime failure causes are demonstrated.
- [] CPU work no longer causes the same main-request stall.
- [] Unused connection/listener objects stop accumulating.
- [] The explanation matches profiles, not just code inspection.

Stretch goals

- Optional: compare two batch sizes for the partitioned workload.

Concepts to read up on

- Node event loop microtask timer I/O
- CPU blocking asynchronous work
- heap snapshot retained size reachability garbage collection

Report back with

- Workload fixtures, profiles, heap comparisons, and changed lifecycle code.
- Before/after measurements and both failure reports.
- Explain why waiting for a promise does not make CPU computation nonblocking.

PROJECT 42 - Keep live rooms responsive under pressure

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 41.

Why now

Correct connection cleanup does not solve a receiver that cannot keep up. Bounded flow control protects fast clients and makes the slow-client policy explicit.

New topics

- Frontend > JS / TS / Node: backpressure and bounded buffering for live connections.

Reinforced topics

- P41 event loop: keep fan-out work from monopolizing the process.
- P41 memory: measure bounded versus growing retained buffers.
- P30 SSE/WebSockets: recover or resynchronize after disconnect.
- P27 metrics: expose connected clients and rejected/dropped updates safely.

The build

A study room remains usable when one connected client reads updates very slowly.

Requirements

1. Run a bounded local test with 20 clients, one intentionally slow consumer, and a recorded event rate.
2. Define a maximum pending buffer or queue per client and choose a documented policy: coalesce replaceable state or disconnect/resynchronize.
3. Keep durable report completion data in the database; dropping a live notification must not lose the underlying result.
4. Measure fast-client update delay and process memory before and after the slow-client protection.
5. Verify reconnect uses the P30 replay/resync path and does not duplicate visible state.
6. Remove all load fixtures from normal startup and document capacity limits of this small test.

PROJECT 42 - Keep live rooms responsive under pressure

Constraints, verification and report

Constraints

- No unlimited in-memory per-client queues.
- No dropping durable business records to protect a transport buffer.
- No claiming 20 simulated clients prove internet-scale capacity.

Break it on purpose

- Pause one client's reads while publishing the test stream. Observe buffered growth on the broken path, apply the limit, and verify bounded memory plus correct resynchronization.

Definition of done

- [] Slow-client behavior follows an explicit bounded policy.
- [] Fast clients remain responsive under the recorded workload.
- [] Memory stops growing with an unbounded pending queue.
- [] Disconnect/reconnect preserves authoritative state.

Stretch goals

- Optional: compare coalescing and disconnect policies for replaceable presence updates.

Concepts to read up on

- backpressure high-water mark bounded queue
- WebSocket buffered output slow consumer
- SSE resynchronization durable state

Report back with

- Flow-control code, client load fixture, and policy notes.
- Memory/update-delay measurements and slow-client failure report.
- Explain which events may be coalesced and which must remain durable.

PROJECT 43 - Reusable dashboard styling

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 42; use small comparison branches for one existing component.

Why now

You now understand plain CSS well enough to evaluate abstractions. Rebuilding one component twice reveals how utility classes and scoped modules change maintenance.

New topics

- Frontend > CSS: Tailwind utility styling.
- Frontend > CSS: CSS Modules and local class scoping.

Reinforced topics

- P3 cascade/layout: preserve the same responsive behavior.
- P11 React: keep component logic and visual variants understandable.
- P16 accessibility: maintain contrast and visible focus.
- P10 bundling: inspect emitted CSS.

The build

The goal card has consistent normal, selected, error, and disabled appearances using two alternative styling approaches.

Requirements

1. Select one goal-card component and freeze its markup behavior and four visual states before comparison.
2. Implement one Tailwind variant and one CSS Modules variant on separate branches or isolated examples.
3. Verify identical content at 360px and 1280px and keyboard focus in both.
4. Change one shared spacing/color token and record how each implementation responds.
5. Measure production CSS output for the same fixture and explain any scope differences.
6. Write a short comparison of readability, variant expression, scoping, and team maintenance; keep both examples for P45.

PROJECT 43 - Reusable dashboard styling

Constraints, verification and report

Constraints

- No component library or full-site restyle.
- No judging only by source line count or an unmatched build.
- No mixing both approaches in the same comparison component.

Break it on purpose

- Add a broad global selector that unintentionally changes the baseline card. Observe where module/utility styling is or is not affected, repair the global rule, and explain the cascade rather than assuming isolation is absolute.

Definition of done

- [] Both implementations satisfy the same four-state contract.
- [] Viewport and accessibility checks pass.
- [] Comparable output and token-change evidence exist.
- [] The cascade failure is explained and fixed.

Stretch goals

- Optional: add one reusable visual variant without changing component behavior.

Concepts to read up on

- Tailwind utilities variants theme tokens
- CSS Modules scoped class names
- cascade inheritance specificity

Report back with

- Two comparison branches/examples, screenshots, and output sizes.
- Token-change notes and cascade failure report.
- Explain one maintenance tradeoff that bundle size cannot capture.

PROJECT 44 - Two alternative component styling approaches

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 43's isolated comparison fixture.

Why now

Preprocessing and runtime styling solve different problems. Reusing the same component prevents a new design from disguising their actual costs.

New topics

- Frontend > CSS: Sass/preprocessors.
- Frontend > CSS: CSS-in-JS using one maintained library.

Reinforced topics

- P3 cascade: understand emitted CSS rather than source syntax alone.
- P43 component styles: preserve the same state and token contract.
- P10 bundlers: inspect transformation and runtime costs.
- P17 hydration: keep server/client initial styles consistent.

The build

The same goal card gains a Sass implementation and a CSS-in-JS implementation for a four-way decision.

Requirements

1. Implement the P43 card with Sass variables/mixins and separately with one maintained CSS-in-JS library.
2. Use the same four states, content, viewport sizes, and tokens as P43.
3. Record generated CSS and any additional client JavaScript for each production build.
4. Verify a server-rendered instance starts with correct styles and no hydration warning; choose a library compatible with the pinned framework.
5. Make the same token and conditional-variant change in both versions and document the edit locations.
6. Keep each variant isolated and preserve functionality/accessibility.

PROJECT 44 - Two alternative component styling approaches

Constraints, verification and report

Constraints

- No framework-wide migration, excessive Sass nesting, or runtime style creation inside unbounded loops.
- No choosing a library without checking compatibility with the installed rendering stack.
- No treating CSS-in-JS as one universal performance profile.

Break it on purpose

- Introduce conflicting style order or a server/client theme mismatch in the fixture. Observe the incorrect initial appearance or warning, repair ordering/initial state, and verify both first paint and interaction.

Definition of done

- [] Two new variants match the shared contract.
- [] Generated output and runtime differences are recorded.
- [] SSR/hydration and accessibility checks pass.
- [] Style-order or theme failure has a demonstrated repair.

Stretch goals

- Optional: compare one build-time CSS-in-JS alternative without migrating the app.

Concepts to read up on

- Sass compilation mixin nesting
- CSS-in-JS style injection static extraction
- SSR style ordering hydration

Report back with

- Sass and CSS-in-JS variants, build evidence, and screenshots.
- Token-change comparison and failure report.
- Explain which styling work happens at build time and which happens at runtime.

PROJECT 45 - Choose and consolidate the styling system

Size: 1 week(s), 10-15 hours total | Mode: Extends Project 44; retain comparison evidence and consolidate the goal-card surface.

Why now

Trying tools is incomplete without making and defending a choice. A small consolidation shows whether the comparison taught transferable judgement.

New topics

- None. This is an integration, repair, or independent transfer assessment.

Reinforced topics

- P43 Tailwind: assess utility composition and state variants.
- P43 CSS Modules: assess scope and local maintenance.
- P44 Sass: assess preprocessing and shared-token changes.
- P44 CSS-in-JS: assess dynamic styling and rendering cost.

The build

StudyTrack uses one chosen approach for the goal card and its immediate list view, with a documented reason for the choice.

Requirements

1. Create a four-way decision matrix using the same correctness, accessibility, token-edit, output, and rendering evidence.
2. Choose one approach for the goal card/list surface; do not migrate unrelated pages.
3. Remove unused runtime styling dependencies from the active build while retaining comparison branches or archived examples.
4. Document how to add a new visual state using the chosen convention.
5. Run existing UI checks and compare a clean build before/after consolidation.

PROJECT 45 - Choose and consolidate the styling system

Constraints, verification and report

Constraints

- No selecting a winner from popularity alone.
- No full application restyle or new design system.
- No leaving two active styling implementations for the same component.

Break it on purpose

- Remove the old implementation, then build from a clean install to expose any hidden import or style dependency. Repair the dependency and verify the card's four states.

Definition of done

- [] The decision is supported by comparable evidence.
- [] One active implementation remains on the chosen surface.
- [] Clean build and visual/accessibility checks pass.
- [] A future contributor has a short styling convention.

Stretch goals

- Optional: have a reviewer add a state using only the convention.

Concepts to read up on

- styling tradeoff decision matrix
- dependency cleanup production build
- maintainability evidence

Report back with

- Decision matrix/ADR, dependency diff, and convention.
- Clean-build results and four-state screenshots.
- Hidden-dependency failure report.
- Explain which future requirement would make you reconsider the choice.

PROJECT 46 - Run the platform on local Kubernetes

Size: 3 week(s), 30-45 hours total | Mode: Extends Project 45; deploy only the web/API slice to a local cluster.

Why now

You already understand containers and a reverse proxy. A small cluster deployment introduces reconciliation and service discovery without moving every stateful dependency at once.

New topics

- Common > Dev environment and deployment: Kubernetes workloads and Services.
- Common > Dev environment and deployment: Kubernetes configuration and Secret references.
- Common > Dev environment and deployment: readiness, liveness, and startup probes.

Reinforced topics

- P21 Docker: deploy existing pinned images.
- P22 infrastructure: document networking and persistence boundaries.
- P20 CI: associate manifests with a known release.
- P24 logging: inspect a failing container's startup.

The build

The existing dashboard/API runs in a local Kubernetes cluster while its existing database remains outside the cluster.

Requirements

1. Choose one local Kubernetes distribution that fits the machine; record resource allocation and versions.
2. Deploy only web/API containers with explicit image digests, labels, selectors, and Services; keep database/search/Kafka in their existing local setup.
3. Supply nonsecret configuration and Secret references through manifests without committing secret values; explain that base64 is not encryption.
4. Implement distinct readiness, liveness, and startup behavior appropriate to the app; liveness must not depend on every optional service.
5. Reach the application through a documented local route and complete a login/create/list journey.
6. Recreate the disposable cluster from the documented manifests while preserving authoritative external database data.

PROJECT 46 - Run the platform on local Kubernetes

Constraints, verification and report

Constraints

- No managed cluster, Helm chart, or importing the entire platform at once.
- No treating a running pod as proof the app is ready.
- No moving databases into ephemeral pods.

Break it on purpose

- Deploy an image reference that cannot start, then a configuration that fails readiness. Distinguish image-pull/startup events from probe failure, repair each, and verify the same user journey.

Definition of done

- [] A clean local cluster reproduces the application slice.
- [] Services, configuration, and probes are explained.
- [] External data survives cluster recreation.
- [] Two different deployment failures are diagnosed.

Stretch goals

- Optional: document the boundary that would need to change for a remote cluster.

Concepts to read up on

- Kubernetes Deployment Pod Service reconciliation
- ConfigMap Secret image digest
- readiness liveness startup probe
- kubectl events describe logs

Report back with

- Manifests, redacted configuration procedure, and cluster setup guide.
- Clean-cluster journey and preserved-data evidence.
- Image/probe failure reports.
- Explain why readiness failure and liveness failure should not always cause the same action.

PROJECT 47 - Recover from bad rollouts and dead pods

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 46.

Why now

A declarative deployment can still roll out a broken release. Bounded traffic and controlled faults show whether replacement and rollback behave as intended.

New topics

- Common > Dev environment and deployment: rolling updates and rollback.
- Common > Dev environment and deployment: resource requests and limits.
- Common > Dev environment and deployment: replica scaling.

Reinforced topics

- P46 Kubernetes: explain desired state versus observed pods.
- P46 probes: control whether a new instance receives traffic.
- P27 metrics: observe latency, errors, and restarts.
- P9 load testing: compare a repeatable workload.

The build

StudyTrack accepts a controlled release and continues serving during one pod failure, with a demonstrated rollback path.

Requirements

1. Run two app replicas and a bounded synthetic read workload; record instance distribution and baseline errors.
2. Configure rollout parameters and probes; deploy a harmless visible version change and measure interrupted requests.
3. Set justified initial CPU/memory requests and limits, then observe throttling or termination under a bounded fixture.
4. Delete one app pod and verify replacement, traffic behavior, and preserved database records.
5. Deploy a bad version that fails readiness; detect the stalled rollout and restore the prior known image.
6. Scale from one to two replicas and compare the fixed workload without claiming universal capacity gains.

PROJECT 47 - Recover from bad rollouts and dead pods

Constraints, verification and report

Constraints

- No relying on automatic rollback unless it is actually configured and demonstrated.
- No increasing resource limits without measuring the cause.
- No stateful database failover in this assignment.

Break it on purpose

- Use a disposable memory-pressure fixture to exceed the container limit. Observe the termination reason and restart metrics, repair the workload or limit based on evidence, and verify the healthy release.

Definition of done

- [] Rollout, pod deletion, bad release, and recovery have measured outcomes.
- [] Resource behavior is explained from events and metrics.
- [] Rollback restores the known image and user journey.
- [] Data remains intact.

Stretch goals

- Optional: evaluate a simple autoscaling policy with the same bounded workload.

Concepts to read up on

- rolling update maxSurge maxUnavailable rollback
- Kubernetes requests limits OOMKilled throttling
- replicas readiness traffic

Report back with

- Updated manifests and image/version record.
- Traffic, restart, and rollout timelines.
- Resource-failure and rollback reports.
- Explain what the two-replica setup still cannot survive.

PROJECT 48 - Make the repository maintainable by someone else

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 47.

Why now

Your own memory can hide missing setup steps and package coupling. A contributor rehearsal tests whether the repository communicates its structure and supports safe change.

New topics

- None. This is an integration, repair, or independent transfer assessment.

Reinforced topics

- P20 Git internals/workflows: resolve history and merge problems without losing work.
- P40 monorepos: enforce package exports and build order.
- P40 type-level TypeScript: protect a shared contract during a real change.
- P40 developer docs: support an independent fresh-checkout contribution.

The build

A new contributor can change one goal-card label and one API contract field, run the right checks, and prepare a traceable release.

Requirements

1. Use a fresh clone and follow only the developer guide to start the minimal app slice and run relevant checks; record every missing instruction.
2. Make two branches that intentionally conflict in a small shared contract, merge them, and resolve using intended behavior plus tests.
3. Inspect the involved commits, trees, and blobs; demonstrate recovering an unreferenced local test commit through the reflog in a disposable clone.
4. Change one derived TypeScript contract and update its legitimate consumers without broad casts or internal-package imports.
5. Document branch/PR/release conventions, package ownership, setup, troubleshooting, and a minimal contribution walkthrough.
6. Prepare a release candidate and verify that its build/check evidence identifies the same commit.

PROJECT 48 - Make the repository maintainable by someone else

Constraints, verification and report

Constraints

- No destructive reset or force-push to erase shared history problems.
- No asking the guide's reader to infer local secrets or undocumented services.
- No adding features beyond the two tiny contribution exercises.

Break it on purpose

- Remove one documented setup variable or package export in the disposable clone. Observe the contributor failure, fix the actual contract/instruction, and rerun the guide from the beginning.

Definition of done

- [] Fresh-clone instructions are complete.
- [] Conflict resolution and local commit recovery preserve intended work.
- [] Type-level checks catch an incorrect consumer.
- [] Release candidate and check evidence agree.

Stretch goals

- Optional: ask a peer to follow the walkthrough and record only observed friction.

Concepts to read up on

- Git merge conflict reflog object graph
- workspace package exports contract type tests
- developer onboarding reproducible contribution

Report back with

- Developer guide, contribution PR, conflict-resolution diff, and type fixtures.
- Fresh-clone findings and disposable reflog recovery evidence.
- Setup-failure report.
- Explain why a commit can exist locally without a branch pointing to it.

PROJECT 49 - Explain and reduce page-loading cost

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 48.

Why now

Framework choices only become meaningful when you can explain the browser's actual work. One public route provides a controlled comparison from network delivery to interactive rendering.

New topics

- Frontend > How browsers work: browser/JS-engine parsing, compilation, execution, and rendering at greater depth.

Reinforced topics

- P17-P18 rendering: compare CSR, SSR, SSG/ISR, streaming, and hydration using existing small routes.
- P5 browsers: relate DOM/style changes to layout and painting.
- P19 caching: separate cold/warm delivery and freshness.
- P16 performance: interpret Lighthouse/Web Vitals evidence honestly.

The build

A public course page becomes measurably faster under a fixed slow-network/CPU profile without losing content or accessibility.

Requirements

1. Choose one public content fixture and record response HTML, network waterfall, main-thread profile, and three-run median Lighthouse results.
2. Compare the existing rendering variants with equivalent visible content; document initial-content, freshness, and interaction differences.
3. Identify one actual bottleneck: render-blocking resource, unnecessary JavaScript, delayed data, or repeated layout work.
4. Make one targeted change and compare cold and warm runs under the same conditions.
5. Show where parsing/compilation/execution appears in the profile and distinguish it from network waiting and layout. Relate one React render/commit from P16 to the browser layout/paint that follows.
6. Retest keyboard access, hydration, cache updates, and private-data boundaries.

PROJECT 49 - Explain and reduce page-loading cost

Constraints, verification and report

Constraints

- No changing hardware/profile/data between before-and-after runs.
- No treating server rendering as proof that hydration costs nothing.
- No deleting needed content or ignoring accessibility to improve a score.

Break it on purpose

- Add a deliberately oversized nonessential bundle or layout-thrashing interaction to the local fixture. Observe the specific waterfall/profile symptom, remove or restructure it, and verify the measured repair.

Definition of done

- [] Bottleneck and improvement are supported by comparable traces.
- [] Rendering choices have route-specific justification.
- [] Cold/warm behavior and freshness are correct.
- [] Accessibility and hydration checks pass.

Stretch goals

- Optional: establish a narrow bundle-size regression budget for the chosen route.

Concepts to read up on

- browser network waterfall critical rendering path
- JavaScript parse compile execute main thread
- layout thrashing hydration streaming
- lab and field performance

Report back with

- Before/after profiles, HTML, waterfall, and median results.
- Changed route/bundle configuration and rendering comparison.
- Failure report.
- Explain which bottleneck a CDN can improve and which it cannot.

PROJECT 50 - Tenant isolation and hostile authorization tests

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 49.

Why now

Authorization errors often appear when roles, ownership, and changing attributes interact. A fixed adversarial matrix tests the policies across every existing entry point.

New topics

- None. This is an integration, repair, or independent transfer assessment.

Reinforced topics

- P14 sessions/CSRF: enforce invalidation and mutation protection.
- P15 OAuth/OIDC/JWT and secrets: reject bad identity claims and verify rotation.
- P14/P31 RBAC/ABAC: combine role, ownership, tenant, and active assignment.
- P7/P12 testing: exercise HTTP and browser behavior against real policy boundaries.

The build

Two organizations use StudyTrack without seeing or modifying each other's goals, reports, resources, or live-room activity.

Requirements

1. Create a synthetic fixture with two organizations, two learners, a mentor, and an administrator; define the expected access matrix before implementation changes.
2. Run at least twelve negative cases spanning ownership, tenant mismatch, role change, assignment expiry, CSRF failure, expired/wrong-audience identity, and logged-out session reuse. Retain the P13 XSS/injection fixtures and repeat allowed/disallowed browser-origin checks.
3. Exercise REST, GraphQL, report download, and live-connection entry points that expose the chosen fixture.
4. Verify role/assignment changes take effect within a documented bound, including cached sessions and open connections.
5. Search logs, traces, build output, and Git-tracked files for a test sentinel secret; rotate the test credential and retest login.
6. Add regression checks for every defect found and update policy documentation.

PROJECT 50 - Tenant isolation and hostile authorization tests

Constraints, verification and report

Constraints

- No widening permissions or removing tests to make the matrix pass.
- No real-user or third-party targets: use owned synthetic fixtures.
- No trusting a frontend tenant selector or token contents without verification.

Break it on purpose

- Warm a permission/session cache, revoke a mentor assignment, and attempt access through an existing connection and a new request. Observe any stale authority, repair invalidation/revalidation, and verify the documented revocation bound.

Definition of done

- [] All matrix outcomes match the written policy.
- [] Every entry point enforces the relevant boundary.
- [] Revocation and credential rotation work.
- [] Sentinel secrets are absent from retained outputs and regression checks pass.

Stretch goals

- Optional: have the mentor add three unseen matrix cases.

Concepts to read up on

- tenant isolation authorization matrix
- revocation stale permissions session cache
- OIDC claim validation secret rotation
- CSRF and persistent connections

Report back with

- Access matrix, fixture, regression tests, and updated policy.
- Redacted identity/secret-scan results.
- Revocation failure report.
- Explain why logout in one browser does not automatically revoke every other active session.

PROJECT 51 - Upgrade APIs without breaking old clients

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 50.

Why now

Consumers can remain on an older contract after the server changes. A narrow compatibility exercise makes versioning, ordering, and deprecation concrete.

New topics

- Backend > APIs: API versioning and compatibility policy.
- Backend > APIs: deterministic pagination contracts.

Reinforced topics

- P6 REST: preserve old request/response behavior.
- P31 GraphQL: evolve fields without breaking an existing query.
- P30 SSE/WebSockets: tolerate additive event fields and resynchronize.
- P39 gateway/BFF: route versions without duplicating business logic.

The build

An old dashboard and an updated dashboard both browse resource history while the API introduces pagination and a new optional field.

Requirements

1. Freeze one existing client fixture and its expected contract; create explicit v1/v2 routes only for the chosen resource list.
2. Add cursor-based pagination with a deterministic tie-breaker and a documented view of concurrent inserts; bound page size and validate cursors.
3. Test empty, one-page, tied-sort-value, final-page, invalid-cursor, and insert-between-pages cases; state the promised consistency semantics.
4. Add one optional GraphQL field while keeping the old query working; do not remove or change existing field meaning.
5. Add a compatible optional field to one SSE/WebSocket event and prove the old consumer ignores it safely.
6. Document deprecation criteria, compatibility tests, and how gateway/BFF routes map to shared application logic.

PROJECT 51 - Upgrade APIs without breaking old clients

Constraints, verification and report

Constraints

- No timestamp-only cursor when timestamps can tie.
- No silently breaking the unversioned/old client or copying whole service implementations for v2.
- No claiming snapshot consistency unless the implementation actually provides it.

Break it on purpose

- Run the frozen client against an intentionally renamed required field and observe its failure. Restore compatibility, then induce a pagination tie to expose duplicate/skipped records and fix the ordering.

Definition of done

- [] Old and new clients pass simultaneously.
- [] Pagination follows documented ordering/consistency rules.
- [] GraphQL and event changes are backward compatible.
- [] Both breaking-change failures are caught by retained fixtures.

Stretch goals

- Optional: add consumer-visible deprecation metadata for the chosen old route.

Concepts to read up on

- API additive breaking change semantic contract
- cursor pagination stable ordering tie-breaker
- GraphQL schema evolution event versioning

Report back with

- v1/v2 contract, cursor logic, old/new client fixtures, and gateway mapping.
- Pagination edge-case results and compatibility checks.
- Failure report.
- Explain what can happen when new records arrive between pages.

PROJECT 52 - Full platform failure-and-recovery exercise

Size: 2 week(s), 20-30 hours total | Mode: Extends Project 51; rehearse the smallest relevant deployment slice.

Why now

Individual recovery drills do not prove you can distinguish interacting failures. A bounded incident rehearsal tests evidence gathering, restoration, and honest operational claims.

New topics

- None. This is an integration, repair, or independent transfer assessment.

Reinforced topics

- P37-P38 Kafka/jobs: recover lag and uncertain publication without wrong totals.
- P46-P47 Kubernetes: distinguish rollout, process, and readiness failures.
- P24-P36 observability: move from alert to logs/traces and verify recovery.
- P22-P23 networking/proxies: isolate DNS, TLS, TCP/UDP, and nginx/Apache behavior.

The build

An operator restores StudyTrack through a documented set of local incidents while preserving known synthetic records.

Requirements

1. Prepare a known dataset, backup, expected totals, service diagram, and restore commands; record the exact active component set.
2. Run three separate incidents: wrong proxy upstream, stopped event consumer, and bad app rollout. Do not combine them before each is understood. During consumer recovery, inspect and safely replay one existing dead-job fixture from P28.
3. For each, record detection time, user symptom, first hypothesis, evidence, repair, recovery time, and data reconciliation.
4. Use one alert, one correlated trace/log path, and one direct network probe in the overall exercise; repeat the tiny TCP/UDP distinction and Apache proxy check from P23. Filter the incident logs by timestamp and instance using the P23 text-processing tools.
5. Restore a disposable database copy and rebuild one derived summary/search projection; compare IDs, versions, and totals.
6. Write a short postmortem with a concrete prevention change and one remaining limitation.

PROJECT 52 - Full platform failure-and-recovery exercise

Constraints, verification and report

Constraints

- No production outages, uncontrolled packet flooding, or faults outside the owned lab.
- No restart-everything diagnosis or reporting recovery before data checks.
- No claiming high availability or recovery objectives beyond measured conditions.

Break it on purpose

- Induce the three listed incidents one at a time. The mentor may choose the order without revealing the fault; diagnose from symptoms, then verify both service health and authoritative/derived data.

Definition of done

- [] All incidents have evidence-backed timelines.
- [] Data reconciliation and disposable restore pass.
- [] Runbooks work without undocumented memory.
- [] Postmortem distinguishes observed facts from assumptions.

Stretch goals

- Optional: combine two already-understood faults and compare diagnostic difficulty.

Concepts to read up on

- incident timeline hypothesis evidence
- recovery time recovery point restore verification
- consumer lag rollout proxy upstream
- postmortem corrective action

Report back with

- Incident pack, runbooks, postmortem, and component/version inventory.
- Alert/log/trace/network evidence and reconciliation results.
- Explain the difference between a service responding again and its data being correct.

PROJECT 53 - Independent helpdesk product

Size: 3 week(s), 30-45 hours total | Mode: Greenfield - start a new repository to prove transfer without StudyTrack's folder structure or domain model.

Why now

A long-running codebase can become familiar enough to hide gaps. A very small second product tests whether you can select and assemble the fundamentals independently.

New topics

- None. This is an integration, repair, or independent transfer assessment.

Reinforced topics

- P11-P18 React/rendering: choose a suitable approach for a small product.
- P8-P9 SQL: design constraints, transactions, and one measured query.
- P14-P15 auth: implement maintained, understood account/session boundaries.
- P7/P12 testing: protect user journeys and hostile access.

The build

A two-role helpdesk where requesters open tickets and agents assign and resolve them.

Requirements

1. Create a new repository with only requester and agent roles, ticket title/body/status, and assignment; no attachments, chat, billing, or organizations.
2. Write schema and acceptance cases before implementing; define allowed status transitions and role/ownership rules.
3. Implement sign-in, ticket creation, requester list/detail, and agent assign/resolve through one complete UI/API/database path. Use the P51 cursor approach for the requester list and test a tied sort value.
4. Use a transaction where needed to prevent inconsistent assignment/status changes; test two simultaneous assignment attempts against your chosen rule.
5. Add unit checks for transitions, database/API integration tests, and two critical E2E journeys including unauthorized access.
6. Prepare synthetic seed data and a fresh-checkout README. Choose only previously learned libraries and justify the rendering/data choices.

PROJECT 53 - Independent helpdesk product

Constraints, verification and report

Constraints

- No copying StudyTrack's application skeleton or complete feature code; reuse your written lessons and small generic utilities you can explain.
- No microservices, Kafka, Elasticsearch, or Kubernetes: this product does not need them.
- No increasing scope to make the portfolio look bigger.

Break it on purpose

- Have two agent clients assign the same unassigned ticket concurrently. Observe the broken race, enforce the documented atomic rule, and verify one valid final assignment and clear feedback to the other client.

Definition of done

- [] The bounded ticket workflow works end to end.
- [] Role/ownership and transition tests pass.
- [] Concurrent assignment is correct.
- [] A fresh checkout runs without StudyTrack or undocumented global configuration.

Stretch goals

- Optional: add a single indexed status filter after the core is complete.

Concepts to read up on

- domain model invariants state transition
- atomic conditional update transaction
- authorization object ownership
- independent vertical slice

Report back with

- New repository, schema, acceptance cases, and check output.
- Short demo of requester and agent workflows.
- Concurrency failure report and fresh-clone evidence.
- Explain three decisions made independently and one where you needed a hint.

PROJECT 54 - Ship and defend the helpdesk release

Size: 3 week(s), 30-45 hours total | Mode: Extends Project 53.

Why now

Independent implementation is only half the transfer test. Releasing and defending the small helpdesk shows whether delivery and diagnosis also transferred.

New topics

- None. This is an integration, repair, or independent transfer assessment.

Reinforced topics

- P20-P23 deployment: ship a pinned, identifiable release and roll it back.
- P24-P36 observability: instrument only what this small product needs.
- P16/P49 performance: measure one real user journey and check accessibility.
- P2/P25/P48 documentation: support setup, operation, and a clear portfolio explanation.

The build

A reviewer can run or visit the helpdesk, understand its tradeoffs, and observe a verified recovery from a seeded failure.

Requirements

1. Package and deploy the helpdesk using the simplest previously learned method that fits the available hosting budget; record image/build and source revision.
2. Add structured request logs, one useful error report or local equivalent, and one health/latency measurement; justify each.
3. Run locked fresh-install checks, manual keyboard checks, and a repeatable ticket-list performance measurement with synthetic data. Include three comparable Lighthouse runs and report their median with the lab limitations.
4. Demonstrate release, rollback, and disposable database restore with record reconciliation.
5. Prepare a concise README, API guide, architecture diagram, limitations, and one failure case study for each portfolio product. Document the ticket API version policy, add one optional response field, and prove the frozen P53 client still works.
6. Complete a mentor review: explain an unfamiliar-looking part of your own code, trace a request, justify a SQL index, and repair one unseen seeded defect without a complete solution.

PROJECT 54 - Ship and defend the helpdesk release

Constraints, verification and report

Constraints

- No installing the entire StudyTrack operations stack by habit.
- No invented production usage or reliability claims.
- No marking a topic solid when the independent explanation or repair fails.

Break it on purpose

- The mentor selects a previously studied deployment, authorization, query, or state defect in the lab. Diagnose it from user symptoms and telemetry, repair it, add the appropriate regression check, and verify the release again.

Definition of done

- [] The independent product is reproducibly delivered.
- [] Operational recovery and data reconciliation pass.
- [] Performance/accessibility evidence has stated conditions.
- [] Both portfolio narratives are accurate and the final review records remaining gaps.

Stretch goals

- Optional: rehearse a ten-minute project discussion tailored to a real junior full-stack vacancy.

Concepts to read up on

- portfolio technical defense
- release rollback restore evidence
- minimal useful observability
- debugging and tradeoff explanation

Report back with

- Release/demo or reproducible deployment package; both portfolio repositories.
- Check results, operations evidence, architecture/API docs, and case studies.
- Unseen-defect report.
- Request a final evidence-based coverage assessment and targeted application/interview plan.

Planned coverage and repetition

Project numbers are planned assessment touchpoints, not evidence of completion. Grouped techniques start at their own relevant brief; routine reuse continues beyond this map.

Area	Syllabus topic	Projects
Common	CLI, paths and shell work	1 -> 2 -> 22 -> 23
Common	Processes, permissions and SSH	22 -> 23 -> 52
Common	Text-processing tools	23 -> 52
Common	HTTP/HTTPS and DNS	4 -> 6 -> 23 -> 52
Common	TCP and UDP	23 -> 52
Common	TLS and certificate diagnosis	4 -> 22 -> 23 -> 52
Common	Git basics and branching/PR workflow	1 -> 4 -> 20 -> 48
Common	Git internals and history recovery	20 -> 48
Common	GitHub Actions, CI/CD and releases	20 -> 21 -> 47 -> 54
Common	Unit tests	2 -> 7 -> 12 -> 53
Common	Integration tests	7 -> 8 -> 50 -> 53
Common	E2E tests	12 -> 14 -> 25 -> 53
Common	Accessibility testing	3 -> 16 -> 49 -> 54
Common	Web Vitals and Lighthouse	16 -> 49 -> 54
Common	Load testing and profiling	9 -> 27 -> 41 -> 49
Common	Node toolchain and package managers	1 -> 7 -> 10 -> 40
Common	Docker and reproducible deployment	21 -> 22 -> 46 -> 54
Common	Kubernetes and infrastructure	22 -> 46 -> 47 -> 52
Common	READMEs and developer docs	2 -> 25 -> 40 -> 48
Common	API documentation and ADRs	7 -> 24 -> 25 -> 54
Frontend	Browser rendering, reflow/repaint and waterfall	5 -> 16 -> 49
Frontend	Browser/JS-engine execution model	16 -> 41 -> 49
Frontend	Semantic HTML, cascade and layout	3 -> 5 -> 11 -> 43
Frontend	Tailwind and CSS Modules	43 -> 45

Planned coverage and repetition

Project numbers are planned assessment touchpoints, not evidence of completion. Grouped techniques start at their own relevant brief; routine reuse continues beyond this map.

Area	Syllabus topic	Projects
Frontend	Sass/preprocessors and CSS-in-JS	44 -> 45
Frontend	JavaScript values, functions and collections	1 -> 2 -> 5 -> 53
Frontend	Closures and lifecycle ownership	5 -> 12 -> 41
Frontend	Modules and package boundaries	2 -> 10 -> 40 -> 48
Frontend	Promises, concurrency and event loop	6 -> 12 -> 41 -> 42
Frontend	Memory, GC and bounded buffering	41 -> 42
Frontend	Node runtime	1 -> 6 -> 21 -> 41
Frontend	React rendering and hooks	11 -> 12 -> 16 -> 49
Frontend	React context, effects, lifecycle and state	12 -> 16 -> 30 -> 53
Frontend	Reconciliation/Fiber and React performance	16 -> 49
Frontend	Linters, formatters, bundlers and transpilers	7 -> 10 -> 40 -> 44
Frontend	TypeScript and type-level work	10 -> 31 -> 40 -> 48
Frontend	Monorepos	40 -> 48
Frontend	CSR, SSR and Next.js	11 -> 17 -> 49 -> 53
Frontend	SSG and ISR	18 -> 19 -> 49
Frontend	Streaming and hydration	17 -> 18 -> 44 -> 49
Backend	SQL schema, joins and transactions	8 -> 9 -> 38 -> 53
Backend	SQL indexes and query plans	9 -> 25 -> 53 -> 54
Backend	NoSQL/document storage	32 -> 34
Backend	Redis as a store and cache	26 -> 27
Backend	REST	6 -> 7 -> 29 -> 51
Backend	GraphQL	31 -> 50 -> 51
Backend	SSE and WebSockets	30 -> 42 -> 50 -> 51
Backend	Webhooks	29 -> 38

Planned coverage and repetition

Project numbers are planned assessment touchpoints, not evidence of completion. Grouped techniques start at their own relevant brief; routine reuse continues beyond this map.

Area	Syllabus topic	Projects
Backend	API versioning and compatibility	51 -> 54
Backend	Pagination	51 -> 53
Backend	Rate limiting	26 -> 39
Backend	Idempotency	29 -> 37 -> 38 -> 52
Backend	BFF and gateways	31 -> 39 -> 51
Backend	Microservices and data ownership	37 -> 38 -> 39 -> 52
Backend	Sessions	14 -> 15 -> 26 -> 50
Backend	JWT and OAuth/OIDC	15 -> 50
Backend	RBAC and ABAC	14 -> 31 -> 50 -> 53
Backend	CORS, CSRF, XSS and injection defenses	13 -> 14 -> 25 -> 50
Backend	Secrets handling	15 -> 24 -> 50
Backend	HTTP caching and CDNs	19 -> 49
Backend	Invalidation and cache failure	19 -> 26 -> 27 -> 50
Backend	nginx, reverse proxying and load balancing	22 -> 23 -> 39 -> 52
Backend	Apache comparison	23 -> 52
Backend	Queues, jobs, retries and dead letters	28 -> 29 -> 34 -> 52
Backend	Kafka and event-driven architecture	37 -> 38 -> 52
Backend	Elasticsearch, indexing and relevance	33 -> 34 -> 52
Backend	Kibana and search analytics	33 -> 34
Backend	Structured logging	24 -> 35 -> 52 -> 54
Backend	Metrics, Prometheus and Grafana	27 -> 36 -> 47 -> 52
Backend	Tracing and OpenTelemetry	35 -> 36 -> 52
Backend	Sentry	24 -> 36 -> 54
Backend	Alerting	36 -> 52

Reference basis

The requirements, project sequence, and estimates are original curriculum design judgments. These official references informed terminology and learning surfaces. They are not a request to read entire documentation sites.

Use each brief's search terms for targeted study. Check version-specific settings against the installed version when the project begins, particularly rendering/cache defaults, identity libraries, and infrastructure configuration.

Node.js learning references

Runtime, modules, asynchronous work, testing, and profiling terminology.

Git: Recording Changes

Working tree, staging, and commit model.

MDN: Your first website

Beginner web structure and small-scope progression.

Next.js App Router documentation

Server/client boundaries and rendering features; check version-specific behavior when starting.

PostgreSQL tutorial

Relational queries, transactions, and database learning foundations.

Kubernetes Basics

Local deployment and operational learning surfaces.

OpenTelemetry concepts

Telemetry signals, traces, and context terminology.

OWASP Cheat Sheet Series

Implementation references for authentication, authorization, and application defenses.

Initial coverage ledger

COVERAGE > untouched | introduced | practiced | solid

All areas remain untouched because no working proof has been reviewed. Assignment, installation, and reading do not advance a status.

untouched: No reviewed proof.

introduced: One correct first implementation and explanation.

practiced: A harder reuse passes, including its relevant failure check.

solid: Independent explanation plus successful transfer or diagnosis after the planned revisits. For broad categories, retain weaker subtopic notes rather than promoting the whole category from one strong example.

Common

fundamentals: untouched | vcs: untouched | testing: untouched | dev_environment/deployment: untouched | documentation: untouched

Frontend

browsers: untouched | css: untouched | js/ts/node: untouched | react: untouched | tooling: untouched | rendering: untouched

Backend

databases: untouched | apis: untouched | auth/security: untouched | caching: untouched | web_servers: untouched | async_workflows: untouched | search/analytics: untouched | observability: untouched

Update this ledger only after reviewing the report-back evidence for the completed assignment.